EFFICIENT EVALUATION OF THE EXPONENTIAL FUNCTION VIA CORDIC-BASED ITERATIVE METHODS: MATHEMATICAL FORMULATION AND FPGA IMPLEMENTATION

Devendra G. Sutar, Nitesh B. Guinde, Samrudhi S. Rege, Arya A. Parab and Shvetali T. Thengil

Electronics & Telecommunication Engineering Department, Goa College of Engineering, Affiliated to Goa University, Farmagudi, Ponda 403401, Goa, India

e-mail: ds@gec.ac.in, ng@gec.ac.in

Abstract: The increasing computational requirements of machine learning and scientific applications have underscored the necessity for efficient hardware implementations of transcendental functions, such as the exponential function e^x, within contemporary processor architectures. This paper delivers a comparative analysis of two hardware implementations within a Microwatt softcore processor: (i) a traditional custom RTL-based implementation of the exponential function and (ii) a custom instruction that utilizes a CORDICbased hardware accelerator for enhanced performance. Post-implementation results on a Xilinx FPGA reveal that the CORDIC based custom instruction delivers marked advantages over traditional designs in terms of power consumption and timing performance. Specifically, our approach achieves a total on-chip power reduction from 0.685 W to 0.30 W, reflecting an approximate 56% decrease, which can be attributed to minimized switching activity and optimized iterative computations. Moreover, the timing analysis indicates a significant enhancement in the worst negative slack, improving from 0.323 ns to 2.804 ns. This change signifies better timing closure and improved frequency scalability. The efficiencies gained from reducing combinational logic and signal activity, alongside advances in pipelining, underscore the benefits of incorporating domain-specific custom instructions within processor architectures. These findings illustrate that the combination of custom instruction extensions with optimized hardware algorithms like CORDIC can offer a robust, power-efficient strategy for accelerating nonlinear functions in both embedded and high-performance computing environments.

1. Introduction

The Microwatt is a lightweight, 64-bit OpenPOWER ISA implementation designed as a soft-core processor for FPGA deployment. Developed as a clean-slate design written in VHDL, it adheres to the Power ISA 3.0 standard, offering a modular, pipelined architecture that is highly amenable to customization. Its open-source nature and adherence to a mature, industry-standard ISA make it an ideal
research testbed for exploring architectural extensions. By providing a transparent framework for the instruction decode and execution stages, Microwatt allows designers to integrate specialized functional units directly into the processor pipeline, bridging the gap between general-purpose computing and
dedicated hardware acceleration.

Within this architectural framework, the implementation of complex mathematical operations becomes a focal point for performance optimization. The exponential function (e^x) is a fundamental mathematical primitive that serves as a cornerstone for a vast array of computational disciplines, including scientific computing, digital signal processing (DSP), machine learning, control engineering, and cryptographic protocols. Representing continuous growth and decay, e^x is essential for modeling complex phenomena in nature and engineering. Its mathematical uniqueness specifically the property where the function is its own derivative makes it indispensable in solving differential equations and performing frequency-domain transformations. As contemporary computational workloads in artificial intelligence and real-time sensor fusion demand higher processing speeds and lower energy

footprints, the efficient hardware realization of transcendental functions like e^xhas transitioned from a theoretical preference to a critical architectural requirement.

In traditional computing paradigms, the exponential function is typically implemented via software libraries executed on general-purpose processors. While software-based routines offer high numerical precision and ease of development, they are inherently limited by sequential execution bottlenecks and significant instruction-cycle overhead. Such limitations are particularly acute in embedded systems and edge-computing devices where stringent real-time constraints and power budgets must be satisfied. Consequently, researchers have turned toward hardware acceleration as a viable means of reducing execution latency and offloading computationally intensive tasks from the main processor core.

Field-Programmable Gate Arrays (FPGAs) have emerged as the premier platform for such hardware acceleration due to their reconfigurable fabric, which allows for the implementation of application-specific logic that exploits fine-grained parallelism and deep pipelining. Unlike fixed-architecture CPUs, FPGAs enable the design of custom execution units that can achieve higher throughput and
deterministic timing. In this study, the exponential function is integrated as a custom instruction within the Microwatt soft-core processor. By extending the processor's instruction set, the exponential computation is moved from the software domain into the hardware execution stage, utilizing a pipelined
structural organization to ensure high clock frequencies and optimal data movement.

The objective of this work is to provide a rigorous comparative analysis of these implementation strategies (CORDIC, Taylor Series, and C-based Math libraries). By assessing each method through the lens of post-synthesis and post-implementation metrics specifically resource utilization (LUTs, FFs, DSPs, BRAMs), timing performance (Worst Negative Slack), and on-chip power consumption this paper identifies the most effective hardware-software partition for exponential computation on the Microwatt processor. Such insights are vital for optimizing next-generation embedded systems that require high-performance transcendental function evaluation within constrained FPGA environments.

2. Microwatt Isa Extension Methodology

The Microwatt open-source POWER ISA soft-core implements its decode-execute pipeline across four key VHDL source files: decode_types.vhdl, predecode.vhdl, decode1.vhdl, and execute1.vhdl. Adding any new custom instruction whether a CORDIC-based exponential accelerator, a Taylor-series compute unit, or a direct hardware exponential module requires a consistent sequence of modifications across these four files. This section presents that sequence as a reusable, implementation-agnostic procedure. The specific functional units (e.g., cordic_unit, taylor_unit, exp_unit) differ across the implementations.

2.1 Pipeline Stage Summary

Table 1 summarises the role of each source file in the decode-execute pipeline and the nature of changes required for any new custom instruction.

Source File	Pipeline Role	Change Required
decode_types.vhdl	Shared vocabulary: enums, ROM record type	Add OP_XXX, INSN_xxx, exp_op-style flag field
predecode.vhdl	First-stage opcode classification ROM	Map chosen primary opcode range to INSN_xxx
decode1.vhdl	Full decode ROM (per-instruction control signals)	Add ROM row: operands, result mux, unit flag
execute1.vhdl	Execution pipeline, functional unit port maps	Instantiate unit; add signals, state bit, case arm, stall logic

2.2 Step-by-Step Instruction Addition Procedure

The following five steps are applied in order for every custom instruction added to Microwatt. Each step is described with its purpose and the precise code pattern, making the procedure directly reusable.

Step 1: Define Types and Flags in decode_types.vhdl

decode_types.vhdl is the shared vocabulary file for the entire decode pipeline. Three additions are made here.

(a) Add operation enumerator to insn_type_t. Insert the new opcode OP_XXX into the insn_type_t enumeration. This is the dispatch key used by execute1.vhdl in its case statement. Placement immediately after OP_ILLEGAL and OP_NOP keeps it adjacent to other special-purpose ALU operations:

```
type insn_type_t is (
    OP_ILLEGAL, OP_NOP,
    OP_XXX,     -- NEW: custom instruction opcode
    OP_ADD, OP_ATTN, ...);
```

Without this enumerator the instruction cannot be routed to its own case arm in execute1 and will fall through to OP_ILLEGAL.

(b) Add per-instruction code to insn_code. Append INSN_xxx at the end of the insn_code enumeration, after all standard and floating-point instructions. The decode ROM is an array indexed by insn_code; the predecode stage produces this value, and decode1 uses it for ROM lookup:

```
    INSN_fsel,
    INSN_xxx     -- NEW: custom per-opcode code
);
```

Appending at the tail avoids disturbing index ranges that guard RB-operand and FPR-access detection.

(c) Add control flag to decode_rom_t. Append a new std_ulogic field to the decode_rom_t record type and set its default to '0' in the decode_rom_init constant. This bit is propagated through the decode record to execute1, where it is tested to launch the functional unit:

```
type decode_rom_t is record
    ...
    xxx_op     : std_ulogic;   -- NEW: '1' for this instruction
    sgl_pipe   : std_ulogic;
end record;
constant decode_rom_init : decode_rom_t := (
    ..., xxx_op => '0', sgl_pipe => '0', ...);
```

A dedicated bit avoids encoding the new instruction inside an existing multi-purpose field, keeping execute1 logic unambiguous.

(d) Register primary opcode in recode_primary_opcode. Add a when branch returning the 6-bit primary opcode assigned to INSN_xxx. Any currently unused POWER ISA major opcode slot may be chosen (e.g., opcode 22 = 6'b010110):

```
when INSN_xxx => return "010110";   -- NEW
```

Without this case the icache recovery logic returns "XXXXXX" for the new instruction, breaking simulation and any opcode-based checks.

Step 2: Map Primary Opcode in predecode.vhdl

predecode.vhdl contains the first-stage classification ROM. It maps a 13-bit address, formed from {primary-opcode[5:0], instruction[10:1]}, to an insn_code value before full decode. One range entry is added to the column_predecode_rom array:

```
2#PPPPPP_00000# to 2#PPPPPP_11111# => INSN_xxx,  -- NEW
others                                => INSN_illegal
```

where PPPPPP is the 6-bit binary representation of the chosen primary opcode.

Claiming all 32 secondary-opcode sub-slots (00000–11111) of the primary opcode gives flexibility for future instruction variants. Without this entry any instruction with the chosen primary opcode is classified as INSN_illegal by predecode and the decode1 ROM is never consulted.

Step 3: Add Decode ROM Row in decode1.vhdl

decode1.vhdl contains the main instruction decode ROM a large array of decode_rom_t records indexed by insn_code. A single row is inserted after the last standard instruction entry and before the others catch-all:

```
INSN_xxx => (
    ALU,  NONE, OP_XXX,
    RA, RB, NONE, NONE, RT,
    ADD, "000",
    '0','0','0','0',
    ZERO,'0',
    NONE,
    '0','0','0','0',
    '0','0',
    NONE,
    '0','0','1',
    '1', NONE),
```

The fields set in this row have the following functions:

Field	Purpose
ALU	Routes instruction to the integer/ALU pipeline
OP_XXX	Execute1 dispatch key; must match the enumerator added in Step 1a
RA, RB / RT	Source operand registers / destination register
xxx_op = '1'	The bit from Step 1c; read by execute1 to start the functional unit
sgl_pipe = '1'	Prevents dual-issue of the multi-cycle instruction

Every field in the decode ROM directly controls a mux or enables a path in downstream logic. Without this row the instruction decodes to the default (OP_ILLEGAL, no output). The xxx_op flag is the critical link between the decode stage and the functional unit instantiation in execute1.

Step 4: Instantiate the Functional Unit in execute1.vhdl

execute1.vhdl must be modified in five coordinated sub-sections to integrate any multi-cycle custom functional unit.

(a) Declare handshake signals. In execute1's declarative region add four signals to wire the port map to the dispatch logic and the state register:

```
signal xxx_result   : std_ulogic_vector(63 downto 0);
signal xxx_start    : std_ulogic;
signal xxx_done     : std_ulogic;
```

(b) Add in-progress state bit. The ex1_t clocked pipeline register already tracks bsort_in_progress and bperm_in_progress. A matching bit is added so the stall logic can hold the stage busy across all N compute cycles:

```
xxx_in_progress  : std_ulogic;    -- NEW in ex1_t record
-- in constant initialiser:
```

```
xxx_in_progress => '0',
```

(c) Instantiate the functional unit entity. Add a generate/port-map block alongside the existing bsort_0 and random_0 instantiations:

```
xxx_unit_0: entity work.xxx_unit
    port map (
        clk    => clk,
        rst    => rst,
        input  => a_in,
        start  => xxx_start,
        result => xxx_result,
        done   => xxx_done
    );
```

Without this port map the functional unit entity never receives a clock or inputs, and xxx_result/xxx_done remain undriven.

(d) Add OP_XXX case arm in the combinatorial execute process. Inside the large instruction-type case statement, insert:

```
when OP_XXX =>
    slow_op          := '1';
    v.e.write_data   := xxx_result;
    v.e.write_enable := '1';
    v.complete       := xxx_done and not v.exception;
```

Setting slow_op='1' prevents retirement on cycle 1. write_data is loaded from xxx_result and complete is held until xxx_done, matching the multi-cycle stall protocol used by the multiplier and bsort.

(e) Wire start/in-progress in the clocked section. In the registered (clocked) part of execute1, two additions complete the handshake. First, at the cycle-1 issue point:

```
v.xxx_in_progress := e_in.xxx_op;      -- latch on cycle 1
v.busy := ... or e_in.xxx_op;           -- stall fetch
xxx_start <= go and e_in.xxx_op;        -- pulse start
```

Second, in the multi-cycle update loop (subsequent cycles while in-progress):

```
if ex1.xxx_in_progress = '1' then
    v.xxx_in_progress := not xxx_done;  -- clear when done
    v.e.valid         := xxx_done;
    v.busy            := not xxx_done;
    v.e.write_data    := xxx_result;
    bypass_valid      := xxx_done;
end if;
```

On each clock cycle while xxx_in_progress='1', the pipeline holds busy='1', preventing decode from advancing. When the functional unit asserts done, write-back data is forwarded and valid is pulsed to commit the result.

Step 5: Verify End-to-End Instruction Flow

After all modifications, Table 2 summarises the expected pipeline flow for the new custom instruction and provides a verification checklist.

Table 2 End-to-End Instruction Flow and Verification Checklist

Stage	File	Expected Behaviour	Verification
Predecode	predecode.vhdl	Primary opcode range match → insn_code = INSN_xxx	Simulate: check insn_code signal at predecode output
Decode	decode1.vhdl	ROM row looked up → OP_XXX, operands, xxx_op='1' forwarded	Simulate: verify decode record fields at decode1 output
Issue	execute1.vhdl	e_in.xxx_op='1' → busy set, xxx_start pulsed, in_progress latched	Waveform: xxx_start high for exactly one cycle
Execute	Functional unit entity	Unit computes result over N cycles driven by start/done handshake	Simulate unit in isolation; verify xxx_result at xxx_done
Complete	execute1.vhdl	xxx_done → xxx_result written to RT, busy cleared, bypass_valid set	Check RT register file write; confirm no pipeline hazard

3. Methodology and Project Overview

The proposed work implements and evaluates different hardware approaches for computing the exponential function (e^x) using a Microwatt soft-core processor integrated within an FPGA design flow. The design is developed and synthesized using the Vivado toolchain, targeting the Arty A7-100T FPGA board as the hardware platform. Initially, the Microwatt processor is configured and extended to support custom computational modules for exponential function evaluation, including CORDIC-based, Taylor series-based, and C math function-based implementations. These modules are either integrated as custom functional units or invoked through software running on the processor, enabling a hardware-software co-design approach. The complete system, including memory interfaces and peripheral connections, is synthesized, implemented, and verified within Vivado. Post-implementation analysis is carried out to evaluate timing and power characteristics. For real-time validation, the computed results of (e^x) are transmitted through a UART interface, allowing observation of outputs on a host system terminal. This setup ensures a complete end-to-end workflow, starting from algorithm design and hardware mapping to on-board execution and output visualization, thereby demonstrating the practical feasibility and performance trade-offs of different exponential computation techniques on FPGA-based embedded systems.

This research evaluates and compares three distinct methodologies for the hardware realization of e^x, each presenting a unique trade-off between resource occupancy and computational performance:

1) The CORDIC (Coordinate Rotation Digital Computer) Algorithm: This method is widely recognized for its hardware efficiency, particularly in multiplier-constrained environments. By operating in hyperbolic mode and utilizing an iterative sequence of shift-and-add operations, the CORDIC algorithm avoids the area-heavy overhead of dedicated multipliers. It offers a low-power, area-efficient solution that is highly suitable for high-speed designs where FPGA DSP slices are scarce.

CORDIC IP Core Configuration

The Xilinx CORDIC v6.0 IP core is instantiated from the Vivado IP Catalog to serve as the primary arithmetic engine for computing the exponential function via the hyperbolic identity e^x = sinh(x) + cosh(x). The IP is configured through the IP Catalog graphical interface before project synthesis. Each configuration parameter is chosen to satisfy the fixed-point data format, numerical precision, and pipeline integration requirements of the custom OP_EXP instruction within the Microwatt processor pipeline.

Table 3: Cordic IP Core Configuration Parameters

Parameter	IP Catalog Label	Value Selected	Justification
Functional Selection	Functional Selection	Hyperbolic	Enables hyperbolic kernel to compute $\sinh(x)$ and $\cosh(x)$for the e^x identity.
Hyperbolic Function	Hyperbolic Function	Sinh and Cosh	Simultaneously generates sinh and cosh on one 64-bit bus, reducing hardware area.
Pipelining Mode	Pipelining Mode	Maximum	Maximizes clock frequency and throughput via registers at every iteration stage.
Iterations	Number of Iterations	16	Provides 16-bit precision (Q1.15) while maintaining low LUT occupancy.
Input Width	Input Width	16	Matches the Q1.15 fixed-point operand format extracted from the processor's 64-bit GPR.
Output Width	Output Width	32	Provides 32-bit resolution per output for high dynamic range in gain-correction arithmetic.
Flow Control	Throttle Scheme	Non-Blocking	Ensures single-cycle instruction execution behavior without pipeline back-pressure stalls.

2) The Taylor Series Approximation: This technique represents the exponential function as a finite polynomial expansion. The Taylor series approach allows designers to precisely control the balance between numerical accuracy and hardware complexity by adjusting the number of expansion terms. While this method provides high precision and lower latency through parallel term calculation, it necessitates significant utilization of Digital Signal Processor (DSP) slices and Look-Up Tables (LUTs) for multi-stage multiplication and accumulation.

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots, \qquad -\infty < x < \infty$$

3) C-Based Mathematical Library Implementation: This approach utilizes standard high-level language functions (such as exp()) translated into hardware through High-Level Synthesis (HLS) or soft-core execution. While this method offers the greatest ease of implementation and leverages highly optimized software-defined numerical precision, it is often not inherently optimized for the physical FPGA fabric. Consequently, it typically results in higher resource utilization and increased power consumption due to the complexity of the underlying floating-point emulation and general-purpose arithmetic structures.

4. Methods

4.1 CORDIC-Based (e^x)

i. Resource Utilization and Implementation Analysis of CORDIC-Based (e^x)

The resource utilization and implementation characteristics of the CORDIC-based exponential function are analyzed using both post-synthesis and post-implementation reports generated in Vivado, as shown in Fig. 1.

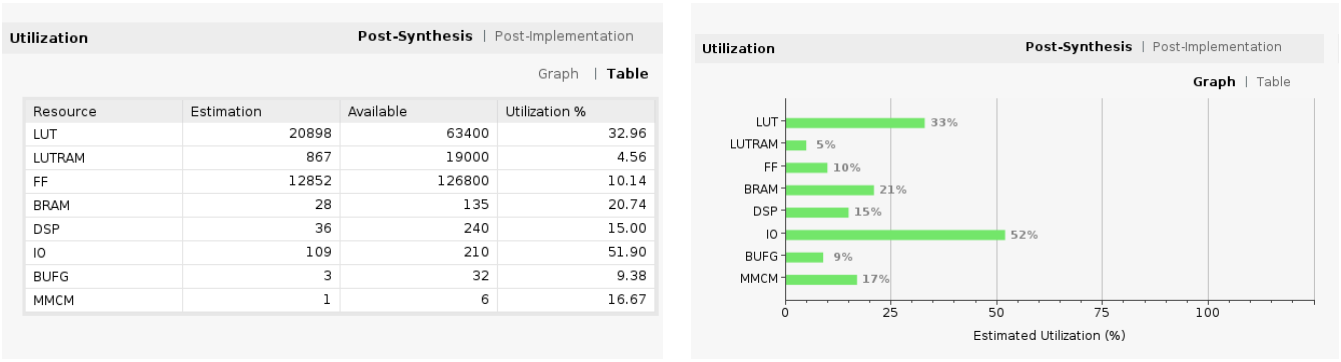
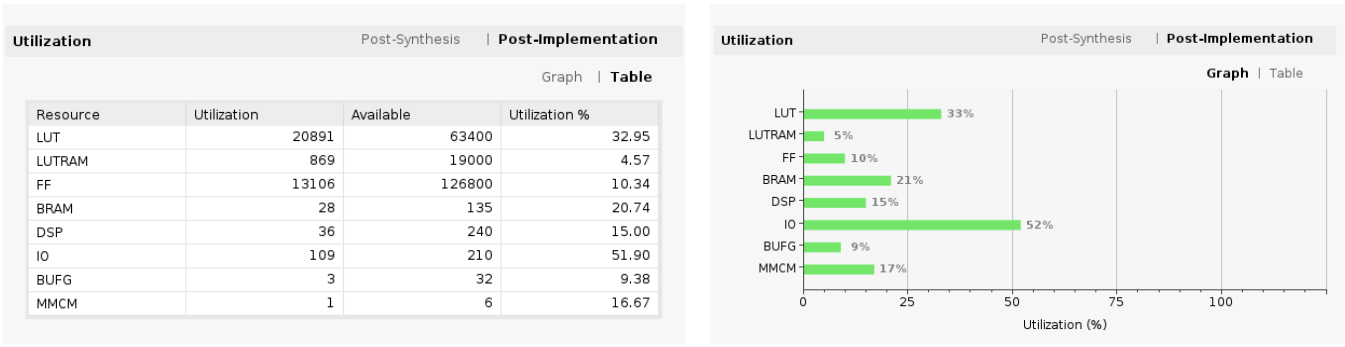


Fig. 1. *Resource utilization of CORDIC-based (e^x) implementation: (a) Post-synthesis utilization (table view), (b) Post-synthesis utilization (graph view)*

Fig. 1(a) and Fig. 2(b) illustrate the post-synthesis resource utilization in tabular and graphical forms, respectively. The design utilizes 20,898 LUTs (32.96%), 12,852 flip-flops (10.14%), and 28 BRAM blocks (20.74%). Additionally, 36 DSP slices (15%) are used, indicating moderate reliance on dedicated arithmetic units. The I/O utilization is relatively high at 51.90%, primarily due to interfacing requirements with peripherals such as UART. Clocking resources, including BUFG (9.38%) and MMCM (16.67%), are efficiently used to maintain stable timing and clock distribution across the design. The graphical representation further highlights that LUTs and I/O dominate the resource usage, reflecting the control logic and communication overhead in the system.



(c) Post-implementation utilization (table view , (d)) Post-implementation utilization (graph view)

Fig. 1(c) and Fig. 1(d) present the post-implementation utilization results, which closely match the synthesis estimates, confirming minimal deviation after placement and routing. The implemented design shows 20,891 LUTs (32.95%) and 13,106 flip-flops (10.34%), indicating slight optimization and redistribution during implementation. BRAM (20.74%) and DSP (15%) usage remain unchanged, demonstrating consistency in memory and arithmetic resource allocation. The stability between synthesis and implementation results validates the efficiency of the design flow and indicates predictable hardware mapping.

In addition to resource utilization, the implementation achieves strong timing performance with a Worst Negative Slack (WNS) of 2.804 ns and zero Total Negative Slack (TNS), confirming that all timing constraints are successfully met. The total on-chip power consumption is measured at 0.3 W, indicating a power-efficient design. The junction temperature remains low at 26.4°C, ensuring safe thermal operation under typical conditions.

ii. Synthesized Design Views and Structural Analysis

The synthesized design of the proposed system is analyzed using multiple visualization views in Vivado, namely the device view, dataflow design view, and schematic representation, as shown in Fig. 2. These views provide complementary insights into placement, logical connectivity, and hierarchical structure of the implemented design.

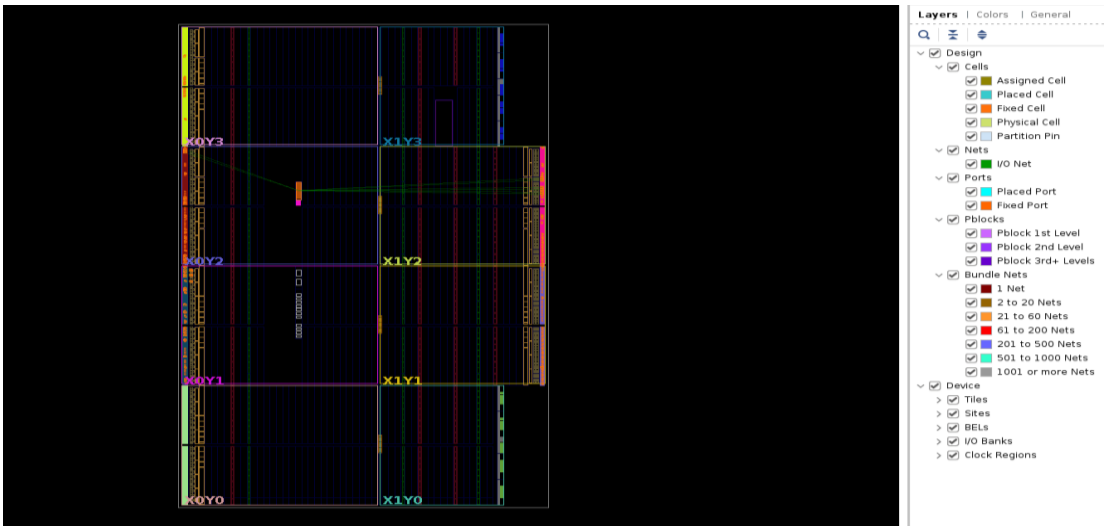
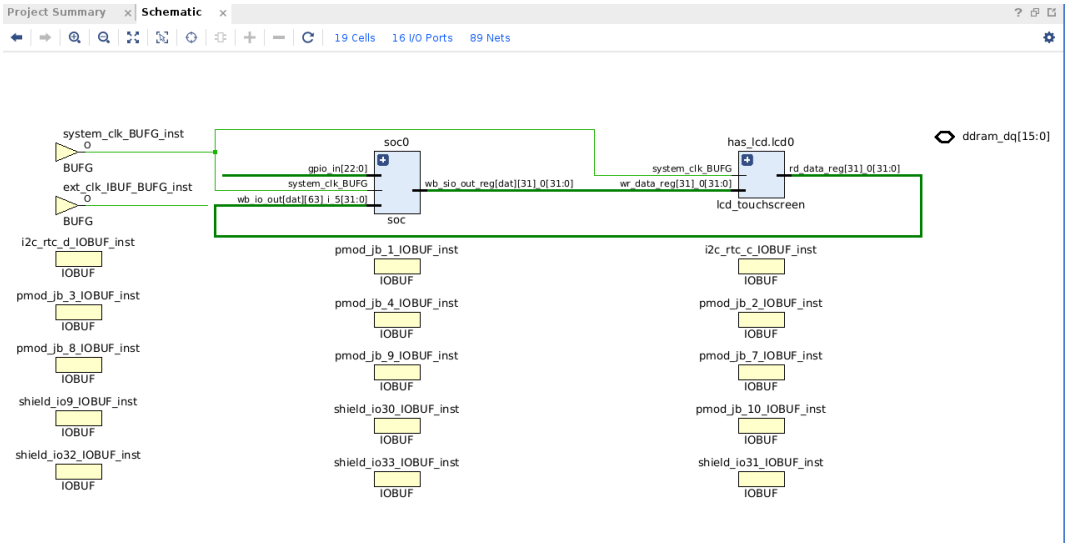


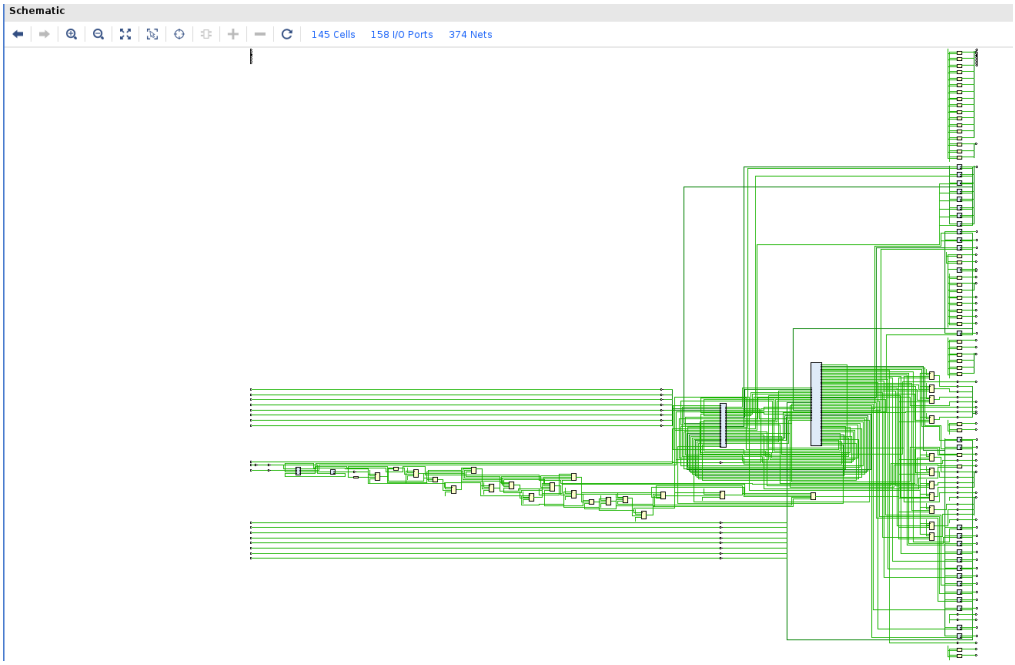
Fig.2 Synthesized design views: (a) Device view showing FPGA resource placement

Fig. 2(a) illustrates the device view of the synthesized design, representing the physical layout of the FPGA fabric. The highlighted regions indicate the placement of logic elements such as LUTs, flip-flops, DSP slices, and BRAMs across different clock regions (X0Y0 to X1Y3). The routing paths between these regions are also visible, demonstrating how signals propagate across the device. The relatively distributed placement of logic resources indicates effective utilization of FPGA fabric, while the routing connections suggest moderate interconnect complexity. This view is critical for understanding spatial resource allocation and identifying congestion or routing bottlenecks.



(b) Dataflow design view illustrating module-level connectivity

Fig. 2(b) presents the dataflow design view, which provides a high-level abstraction of the system architecture. The Microwatt-based SoC core (soc0) is shown interfacing with peripheral modules through structured signal paths such as wb_io_out, wb_sio_out_reg, and rd_data_reg. Clock buffers (BUFG) and I/O buffers (IOBUF) are also visible, ensuring proper clock distribution and external interfacing. The dataflow representation highlights the modular organization of the design, where computation, control, and I/O operations are interconnected through well-defined buses. This abstraction simplifies the understanding of system-level data movement and functional partitioning.



(c) Schematic view representing detailed logical interconnections.

Fig. 2(c) shows the synthesis schematic, which provides a detailed gate-level or RTL-level connectivity of the entire design. The schematic includes a large number of interconnected logic elements, demonstrating the complexity of the implemented system. The dense network of connections reflects the integration of the exponential computation unit within the Microwatt processor framework. This view is particularly useful for debugging, verifying signal paths, and analyzing logical dependencies between modules. The presence of multiple hierarchical blocks and extensive net connections indicates a deeply integrated design with significant control and data interactions.

iii. Implemented Design Views and Structural Analysis

The implemented design of the proposed Microwatt-based system is analyzed using multiple visualization views in Vivado, namely the device view, dataflow design view, and implementation schematic, as shown in Fig. 3. These views provide complementary information regarding physical placement, module-level connectivity, and detailed logical realization after place-and-route.

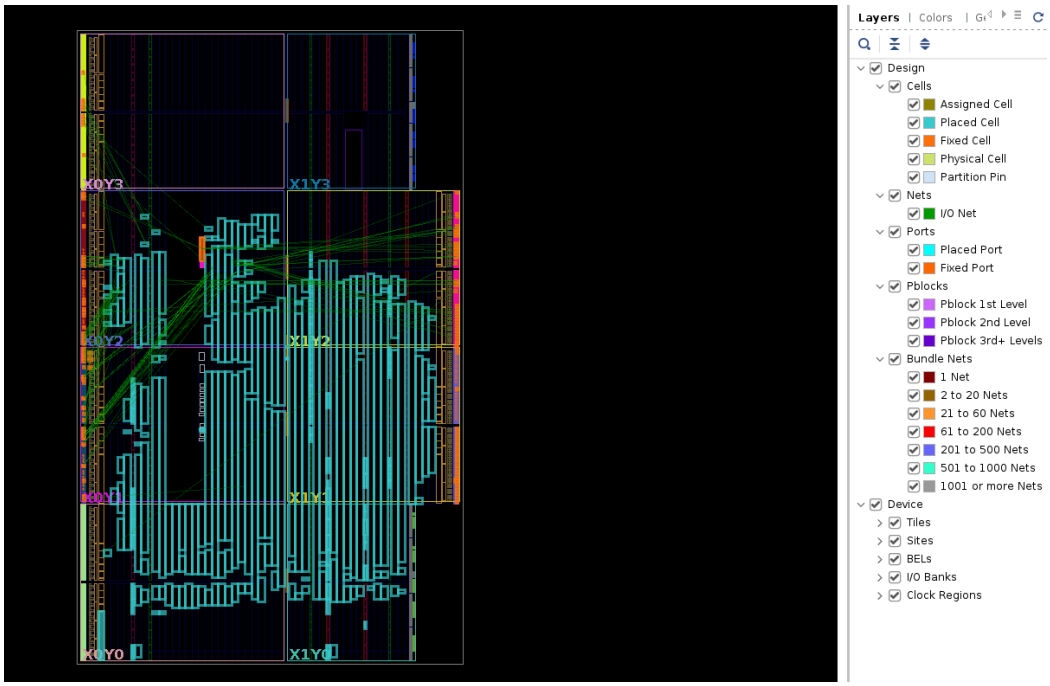
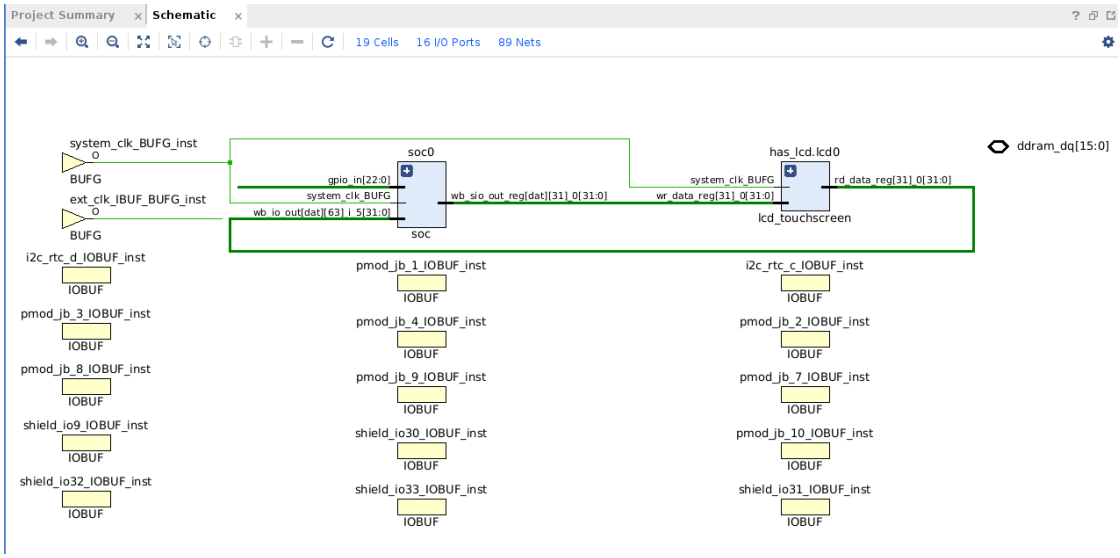


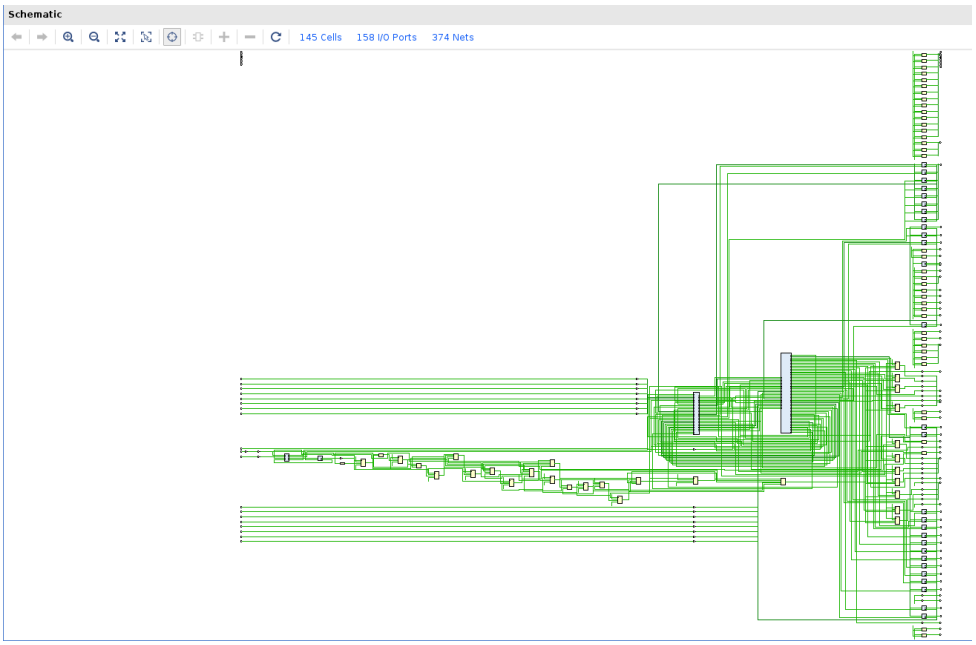
Fig. 3. Implemented design views: (a) Device view showing FPGA resource placement after implementation

Fig. 3(a) illustrates the device view of the implemented design, showing the placement of logic resources across the FPGA fabric. The occupied regions indicate the distribution of LUTs, flip-flops, routing resources, and I/O elements within different clock regions of the device. The placement appears concentrated within a limited area of the FPGA, indicating efficient resource utilization and reduced routing overhead. The visible routing paths demonstrate successful interconnection of the implemented logic while maintaining placement locality.



(b) Dataflow design view illustrating implemented module connectivity

Fig. 3(b) presents the implementation dataflow view, providing a high-level representation of the system architecture after implementation. The Microwatt-based SoC core (soc0) is connected to peripheral and interface modules through structured buses and registered signal paths. Clock distribution is handled through dedicated BUFG resources, while I/O communication is achieved using IOBUF instances. This view highlights the modular organization of the design and confirms proper integration of processing, control, and external interface components.



(c) Implementation schematic showing detailed logical interconnections

Fig. 3(c) shows the implementation schematic, which depicts the detailed logical connectivity of the realized design. The schematic contains multiple interconnected logic blocks, signal nets, and hierarchical structures representing the final implemented hardware. The dense interconnection network reflects the complexity of the

integrated processor subsystem and associated functional units. This representation is useful for validating signal propagation paths, verifying module interactions, and ensuring correctness of the implemented architecture.

iv. Timing, Power, and I/O Placement Analysis

The implemented CORDIC-based exponential (e^x) computation unit was evaluated in Vivado using post-implementation timing, power, and I/O placement analyses, as shown in Fig. 4. These analyses provide insights into the operational performance, energy efficiency, and physical resource organization of the FPGA implementation.

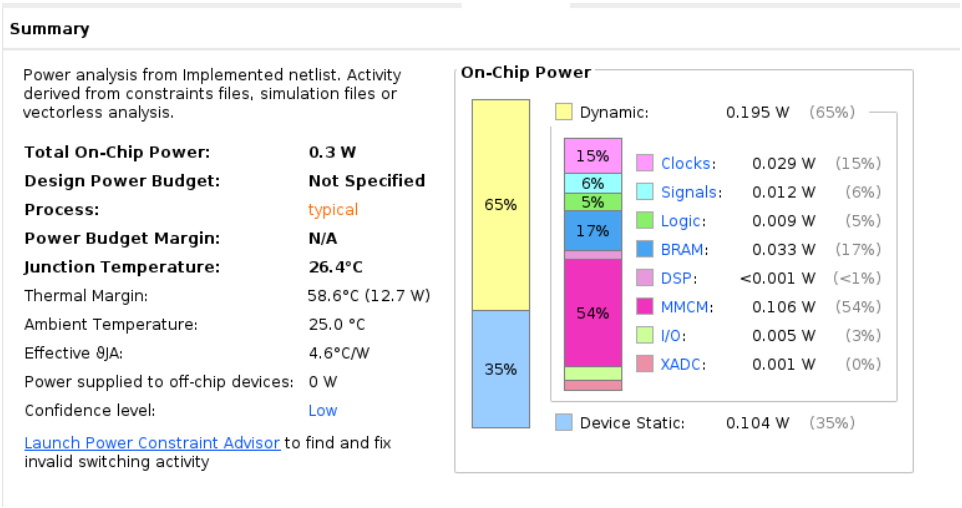
```
| Design Timing Summary
| ---------------------
------------------------------------------------------------------------------------------------

    WNS(ns)      TNS(ns)  TNS Failing Endpoints  TNS Total Endpoints      WHS(ns)      THS(ns)  THS Failing Endpoints  THS Total Endpoints     WPWS(ns)     TPWS(ns)  TPWS Failing Endpoints
TPWS Total Endpoints
    -------      -------  ---------------------  -------------------      -------      -------  ---------------------  -------------------     --------     --------  ----------------------
--------------------
      2.804        0.000                      0                37740        0.012        0.000                      0                37740        3.000        0.000                      0
14016


All user specified timing constraints are met.
```

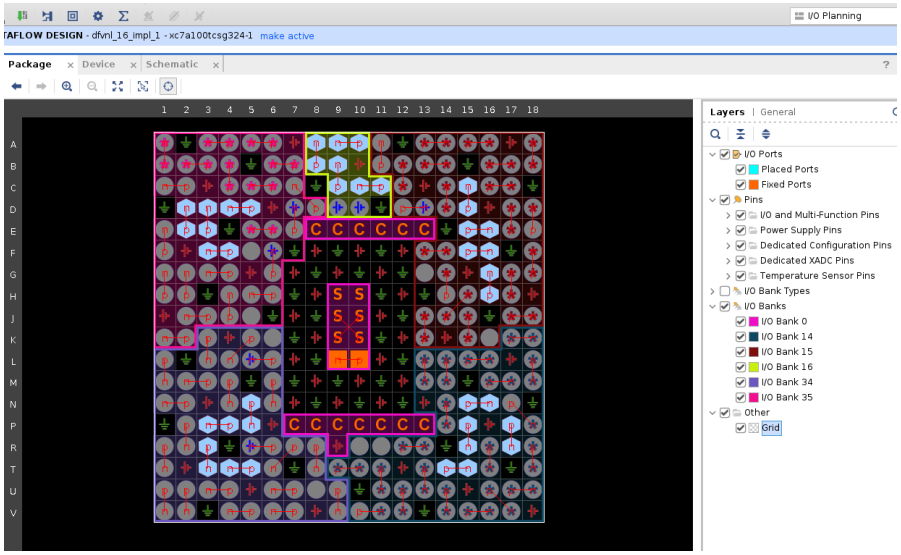
Fig. 4 Implementation analysis of the CORDIC-based e^x computation unit: (a) Timing summary showing successful timing closure

Fig. 4(a) presents the timing summary of the implemented design. The timing report indicates that all user-defined timing constraints are satisfied with positive slack values across setup, hold, and pulse-width checks. A Worst Negative Slack (WNS) of 2.804 ns and a Worst Hold Slack (WHS) of 0.012 ns were achieved, while no failing endpoints were reported. The absence of timing violations demonstrates that the CORDIC-based architecture meets the required performance specifications and can operate reliably at the target clock frequency. The positive timing margins further indicate efficient pipelining and balanced routing throughout the design.



(b) Power report indicating on-chip power consumption

Fig. 4(b) shows the power analysis obtained after implementation. The total on-chip power consumption is estimated at approximately 0.30 W, comprising 0.195 W of dynamic power and 0.104 W of static power. The clock management circuitry contributes the largest portion of dynamic power, followed by BRAM and clock network resources. The relatively low power consumption highlights the efficiency of the CORDIC algorithm, which primarily utilizes shift-and-add operations instead of computationally expensive multipliers, making it suitable for energy-constrained FPGA applications.



(c) I/O placement view illustrating FPGA resource and pin allocation

Fig. 4(c) illustrates the I/O placement view of the implemented design. The figure shows the distribution of input/output pins and logic resources across the FPGA package. The placement demonstrates effective utilization of available I/O banks while maintaining compact routing paths between computational and interface modules. The balanced allocation of resources reduces routing congestion and contributes to the positive timing results observed in the implementation report. Furthermore, the organized placement of clock and signal routing resources ensures stable communication between the exponential computation unit and external interfaces.

4.2 Taylor based (e^x)

v. Resource Utilization and Implementation Analysis of Taylor Series Based (e^x)

The hardware resource consumption of the proposed system was evaluated at both the post-synthesis and post-implementation stages to assess the efficiency of the Taylor series-based exponential computation unit integrated within the Microwatt soft-core. The utilization metrics for Look-Up Tables (LUTs), Flip-Flops (FF), Digital Signal Processors (DSP), and Block RAM (BRAM) are detailed in Fig. 5.

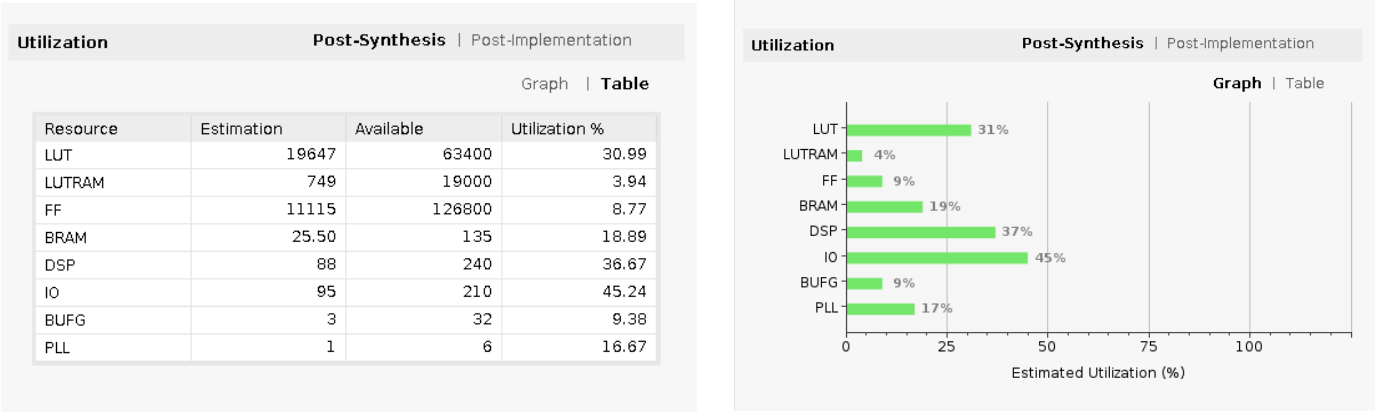
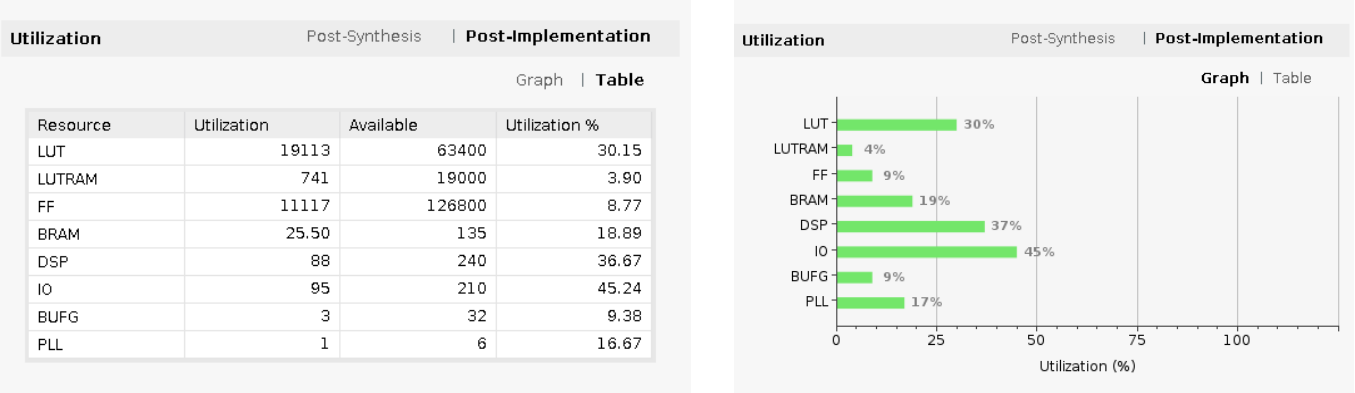


Fig. 5. FPGA Resource Utilization Reports: (a) Post-synthesis detailed resource table (b) Post-synthesis graphical summary

Fig. 5(a) and (b) present the post-synthesis utilization estimates. The design consumes 19,647 LUTs (30.99%) and 11,115 FFs (8.77%) of the available FPGA fabric. A significant observation is the utilization of 88 DSP slices (36.67%), which is attributed to the multiple high-precision multiplication operations required for the higher-order terms in the Taylor series expansion. The BRAM usage stands at 18.89%, providing the necessary memory

overhead for the Microwatt processor's instruction and data storage. The I/O utilization is recorded at 45.24%, reflecting the communication overhead between the processor
core and external test interfaces.



Utilization | Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	19113	63400	30.15
LUTRAM	741	19000	3.90
FF	11117	126800	8.77
BRAM	25.50	135	18.89
DSP	88	240	36.67
IO	95	210	45.24
BUFG	3	32	9.38
PLL	1	6	16.67

(c) Post implementation detailed resource table, and (d) Post-implementation graphical summary

Fig. 5(c) and (d) illustrate the post-implementation utilization, representing the final resource mapping after place-and-route optimizations. The LUT usage slightly decreased to 19,113 (30.15%), indicating that the Vivado implementation tools successfully performed logic pruning and optimization by merging redundant logic gates without affecting functional integrity. The DSP and BRAM counts remained constant, confirming that these dedicated hardware primitives were efficiently mapped during synthesis. The consistency between synthesis and implementation results suggests a stable design with predictable routing requirements and minimal congestion risk.

vi. Synthesized Design Views and Structural Analysis

The structural integrity and logical organization of the proposed e^x computation system were analyzed post-synthesis using Vivado's advanced visualization tools. These views, comprising the schematic, dataflow, and physical device representations, provide a multi-layered perspective on how the Taylor series algorithm is integrated into the Microwatt SoC framework.

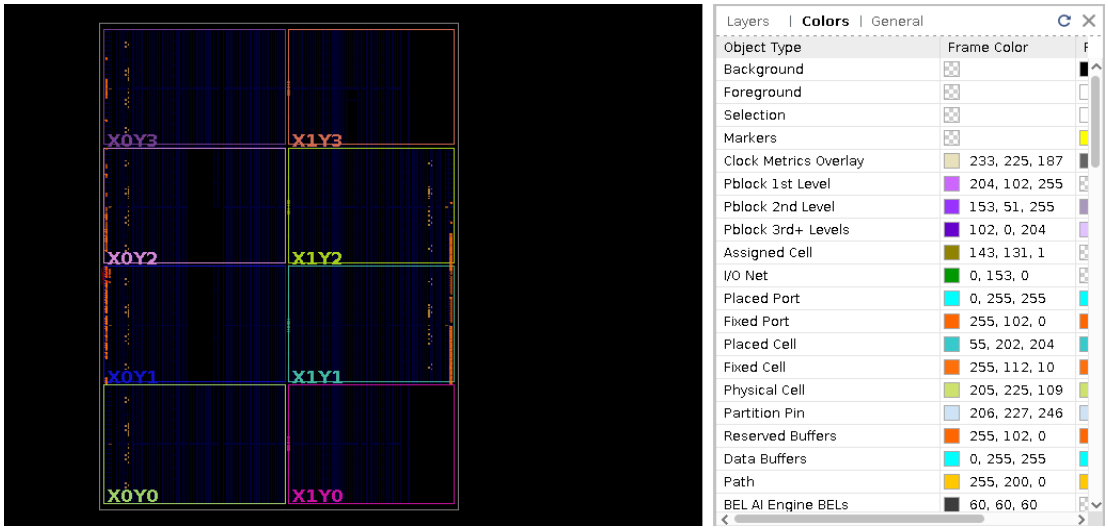
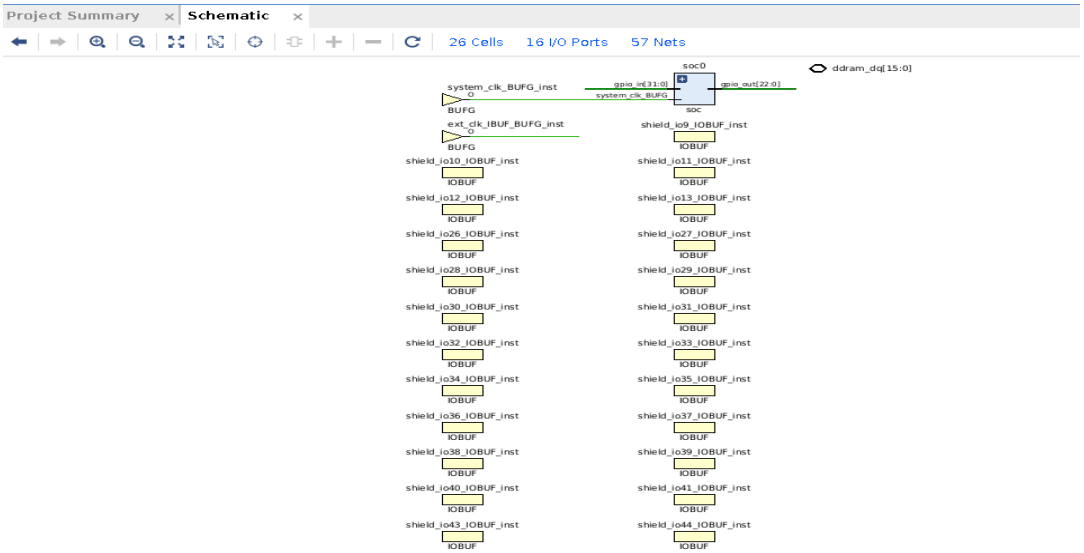


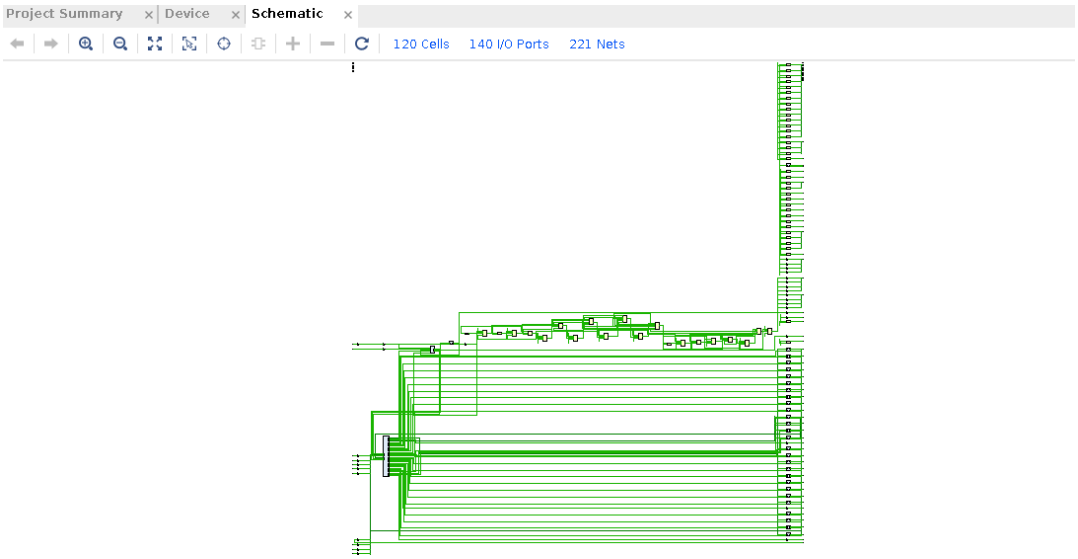
Fig. 6. Implementation Structural Analysis: (a) Device view showing the physical placement and floorplanning on the Artix-7 FPGA fabric.

Fig. 6(a) displays the synthesized device view, representing the preliminary mapping of logical resources onto the physical Artix-7 FPGA fabric. The logic density is distributed across clock regions X0Y0 to X1Y3. The highlighted blue regions represent the spatial allocation of Look-Up Tables (LUTs) and registers, while the vertical orange structures indicate the placement of I/O banks and global clocking resources. The relatively uniform distribution of resources suggests that the synthesis tool has effectively minimized routing congestion, which is critical for achieving the timing closure necessary for real-time exponential function evaluation.



(b) Dataflow design view illustrating system-level module connectivity and I/O buffering

Fig. 6(b) illustrates the dataflow design view, providing a high-level architectural abstraction. This view highlights the modularity of the design, where the central Microwatt core (soc0) interfaces with a series of Input/Output buffers (IOBUF) labeled as shield_io. The inclusion of system_clk_BUFG_inst and ext_clk_IBUF_BUFG_inst ensures synchronized data movement across the computation pipeline. The hierarchy browser confirms the resource footprint within this view, specifically noting the presence of 88 DSP slices and 28 Block RAMs, which are essential for the high-precision arithmetic required to maintain accuracy in the e^x calculation.



(c) Detailed schematic view showing gate-level netlist and hierarchy

Fig. 6(c) presents the synthesis schematic view, which details the gate-level connectivity of the entire system. The netlist window indicates a complex hierarchy consisting of 120 leaf cells and 221 nets. Key top-level modules such as nodram.clkgen (clock generator), nodram.reset_controller, and the primary processor core soc0 are visible. The schematic illustrates the dense interconnection of logic gates and specialized primitives required to facilitate the iterative multiplications and additions inherent in the Taylor series expansion. This view confirms that the exponential unit is correctly interfaced with the processor's internal bus and control logic.

vii. Implemented Design Views and Structural Analysis

After synthesis, the design underwent physical implementation and optimization to ensure hardware efficiency. The implementation views in Vivado confirm the final hardware instantiation of the e^x computation unit and the Microwatt core on the FPGA substrate.

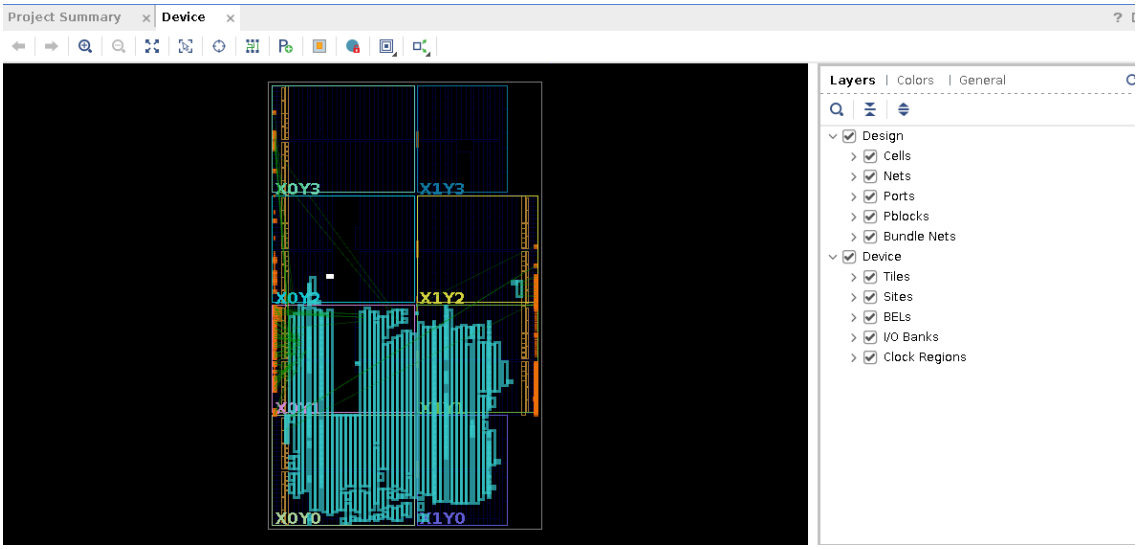
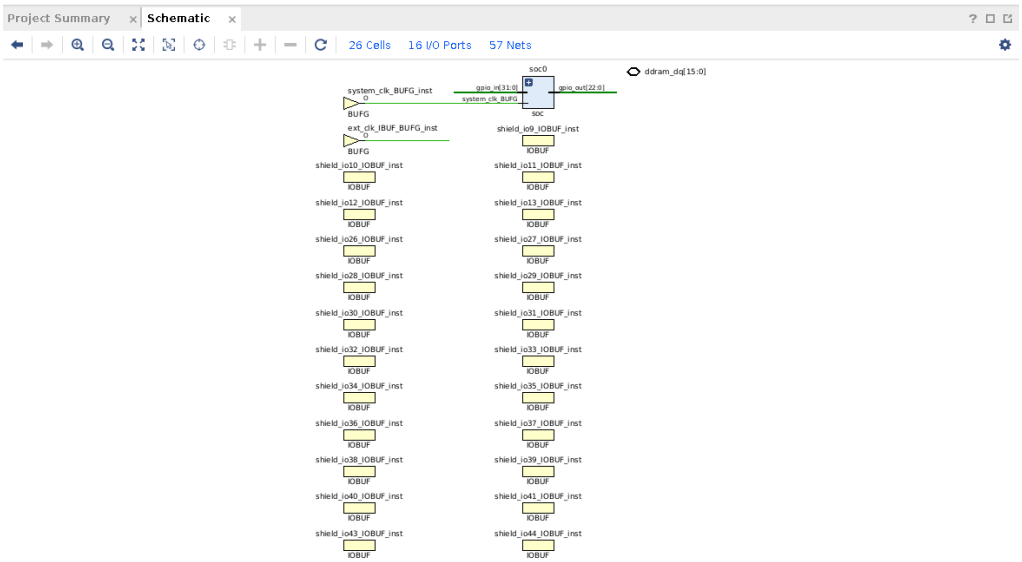


Fig. 7. Post-implementation design representations: (a) Device layout view showing physical placement and routing across FPGA clock regions

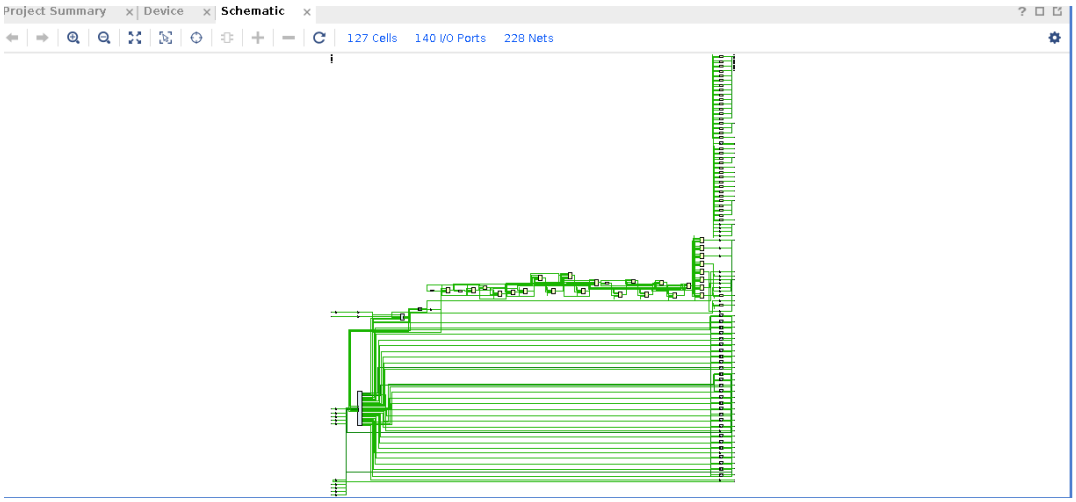
Fig. 7(a) showcases the implemented device view, detailing the physical layout and routing on the Artix-7 FPGA fabric. The teal-colored regions indicate the final placement of logic slices across clock regions X0Y0, X1Y0, X0Y1, and X1Y1. The concentration of logic in the lower regions suggests a successful effort by the implementation tools to minimize wire lengths between the processor and the DSP slices used for Taylor series multiplication. The green routing lines represent the actual physical wires established during the routing phase. The absence of high-density congestion markers confirms that the routing resources were sufficient to handle the data-heavy paths required for floating-point or fixed-point exponential approximations.



(b) Dataflow view representing I/O buffer mapping and clock distribution

Fig. 7(b) illustrates the implementation dataflow design, which provides high-level visualization of the physical I/O mapping. This view emphasizes the top-level connectivity of the soc0 core with its external environment. A total of 26 top-level cells and 16 I/O ports are visible, specifically showing the instantiation of multiple shield_io_IOBUF_inst modules. These buffers serve as the physical interface between the internal logic of the exponential unit and the FPGA's package pins. The presence of system_clk_BUFG_inst highlights the finalized

clock distribution network, which is critical for maintaining synchronous operation during high-speed iterative calculations.



(c) Final physical schematic with cell and net statistics

Fig. 7(c) presents the implementation schematic view, representing the design after logical primitives have been mapped to physical hardware resources. The report indicates a total of 127 cells, 140 I/O ports, and 228 nets. Unlike the synthesis schematic, this view reflects the finalized physical connectivity, where abstract logic gates are replaced by specific FPGA primitives. The dense network of horizontal and vertical signal paths illustrates the complex interconnection between the Taylor series arithmetic units and the Microwatt processor's register file and control unit.

viii. Timing, Power, and I/O Placement Analysis

A post-implementation analysis was executed to verify the operational integrity of the e^x Taylor series module. Key performance indicators included an assessment of timing margins to prevent data corruption at high speeds, a power analysis to quantify the hardware's efficiency, and a validation of the FPGA pin
mapping to ensure seamless physical connectivity. By examining these parameters, we ensured that the Taylor series implementation is not only mathematically accurate but also physically optimized for the target hardware.

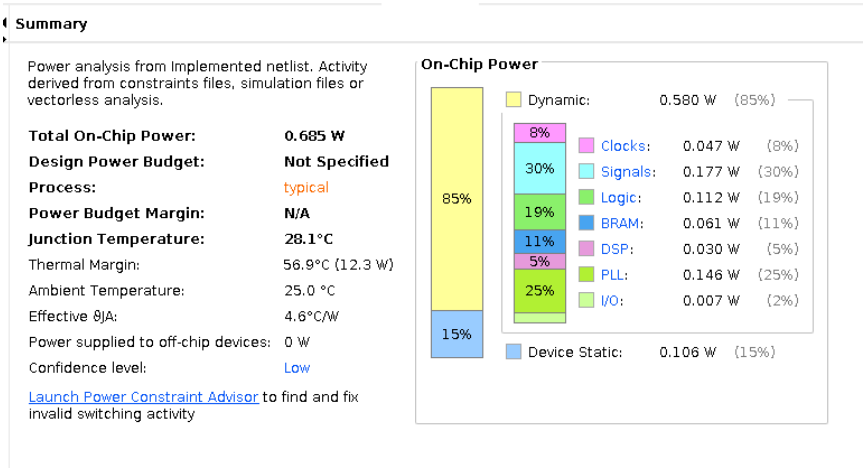
```
| Design Timing Summary
| ---------------------

    WNS(ns)      TNS(ns)  TNS Failing Endpoints  TNS Total Endpoints      WHS(ns)      THS(ns)  THS Failing Endpoints  THS Total Endpoints     WPWS(ns)     TPWS(ns)  TPWS Failing Endpoints
TPWS Total Endpoints
    -------      -------  ---------------------  -------------------      -------      -------  ---------------------  -------------------     --------     --------  ----------------------
-------------------
      0.323        0.000                      0                33010        0.012        0.000                      0                33010        3.000        0.000                      0
11890


All user specified timing constraints are met.
```

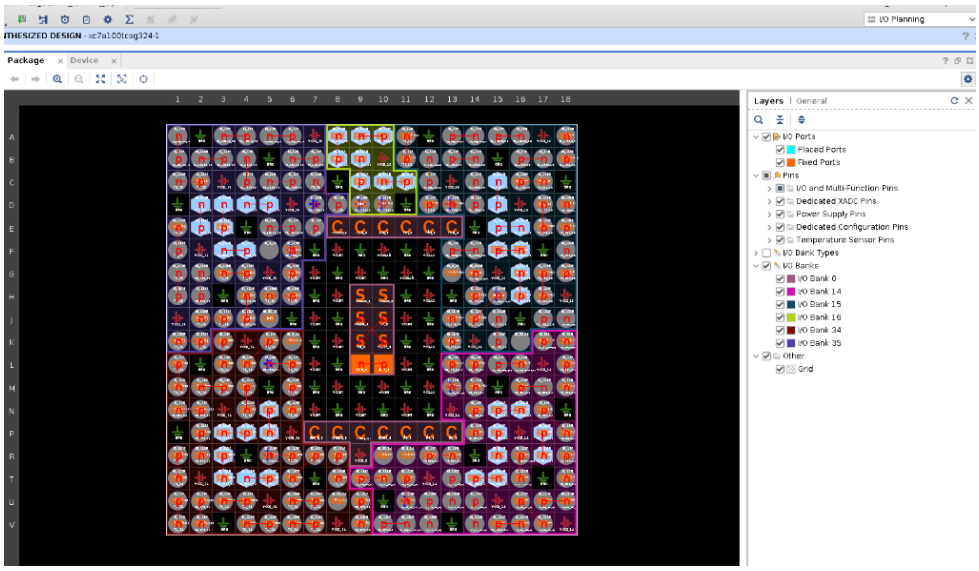
Fig. 8. Performance and Physical Metrics: (a) Post-implementation timing summary

Fig. 8(a) presents the design timing summary, which confirms that the implementation meets all user-specified timing constraints. The design achieves a Worst Negative Slack (WNS) of 0.323 ns and a Worst Hold Slack (WHS) of 0.012 ns. The positive slack values across 33,010 total endpoints indicate that the
critical paths primarily those involved in the high-order Taylor series multiplications are optimized to complete within the required clock period. The Total Negative Slack (TNS) and Total Hold Slack (THS) are both 0.000 ns, signifying zero timing violations. This successful timing closure ensures that the e^x computation unit can operate at its target frequency without data corruption or metastable states.



(b) Power summary detailing dynamic and static consumption

Fig. 8(b) illustrates the power analysis report, which indicates a total on-chip power consumption of 0.685W. The power profile is dominated by dynamic power (0.580 W, representing 85% of total consumption), which is typical for a processor-based arithmetic unit operating at high frequencies. A detailed breakdown shows that the Phase-Locked Loop (PLL) and Clock Manager account for 25% of the dynamic power, while signal switching and logic operations contribute 30% and 19% respectively. The relatively low DSP power (5%) suggests that while the Taylor series calculation is computationally intensive, the iterative nature of the algorithm allows for efficient resource toggling without excessive thermal overhead. The static power remains low at 0.106 W (15%), ensuring a junction temperature of 28.1°C, well within the safe operating range.



(c) Package view illustrating physical I/O pin placement

Fig. 8(c) provides the package view for I/O placement, showcasing the physical pin assignments across the Artix-7 package banks. The colored regions represent different I/O banks used for interfacing the Microwatt core with external peripherals and testing interfaces. The Fixed Ports status indicated in the legend confirms that the implementation adheres to the predefined constraints file, ensuring that the exponential calculation results can be reliably transmitted via the designated pins. The systematic
distribution of used pins across multiple banks helps in mitigating Simultaneous Switching Noise, which is crucial for maintaining signal integrity in high-precision transcendental function outputs.

4.3 C-code Math Function Based (e^x)

ix. Resource Utilization and Implementation Analysis C-Based Math Implementation

The hardware resource utilization for the direct C-based implementation of the exponential function was evaluated to determine the architectural impact of using standard library-style function calls on the Microwatt soft-core. Unlike iterative hardware-centric algorithms, this approach relies on the processor's
ability to execute software-defined arithmetic routines. The resource metricsare summarized in Fig. 9.

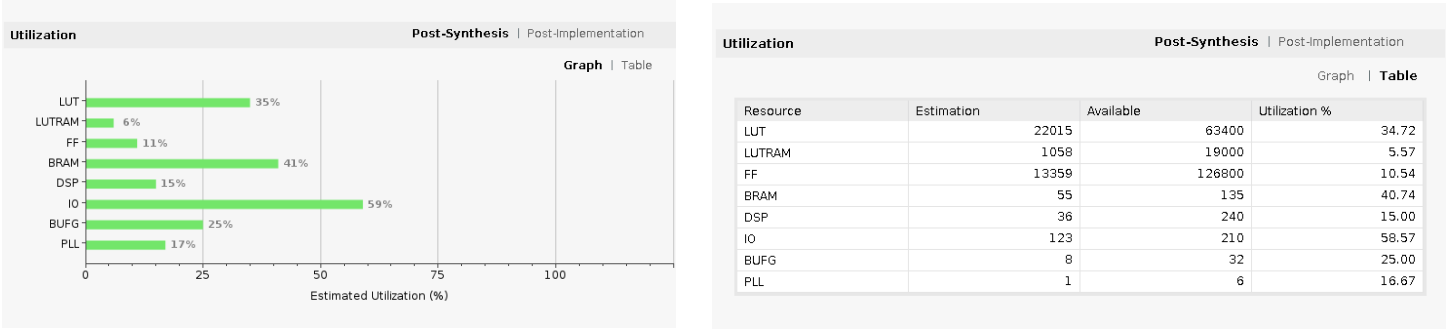
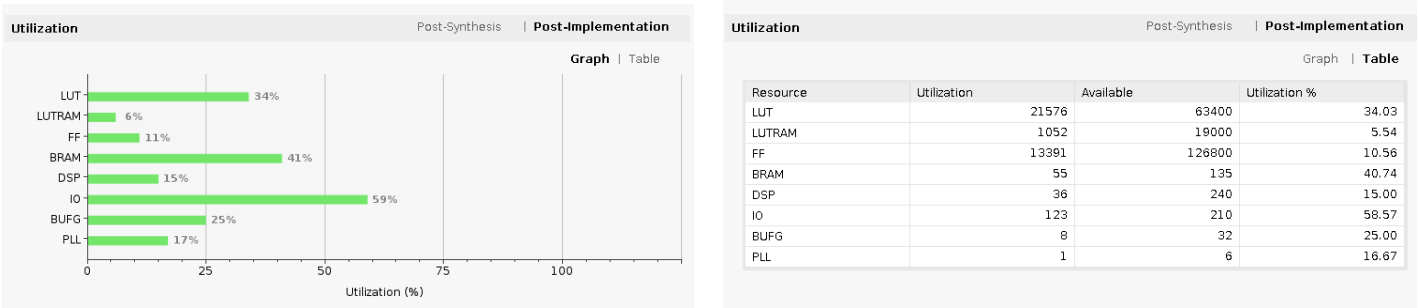


Fig. 9. Resource Utilization for C-based Math Implementation: (a) Post-synthesis graphical summary, (b) Post-synthesis detailed resource table

Fig. 9(a) and (b) illustrate the post-synthesis utilization estimates. The design requires 22,015 LUTs (34.72%) and 13,359 FFs (10.54%). A notable observation in this implementation is the high Block RAM (BRAM) usage, reaching 40.74% (55 instances). This elevated BRAM consumption is likely due to the larger instruction memory footprint required to store the standard C math libraries and the potential use of internal lookup tables (ROM) to assist the math functions. Conversely, the DSP utilization is relatively low at 15.00% (36 slices), suggesting that this software-based approach utilizes the Microwatt's general-purpose ALU more heavily than dedicated hardware multipliers.



(c) Post-implementation graphical summary, and (d) Post-implementation detailed resource table

Fig. 9(c) and (d) present the post-implementation utilization, representing the final resource mapping on the Artix-7 FPGA fabric. The results remain highly consistent with the synthesis estimates, with a marginal reduction in LUT usage to 21,576 (34.03%) and a slight optimization in LUTRAM to 5.54%. The stability of the BRAM and DSP counts (remaining at 40.74% and 15.00% respectively) indicates that the physical implementation tools found a direct mapping for these resources with minimal need for further logic redistribution. The high I/O utilization (58.57%) is consistent with the processor's requirement for external data communication and debugging interfaces.

x. Synthesized Design Views and Structural Analysis (C-based Implementation)

The structural complexity of the software-defined e^x implementation was analyzed through Vivado's synthesis visualization tools. This implementation relies on the processor's ability to execute standard C math libraries, which results in a distinct architectural footprint compared to hardware-optimized iterative methods.

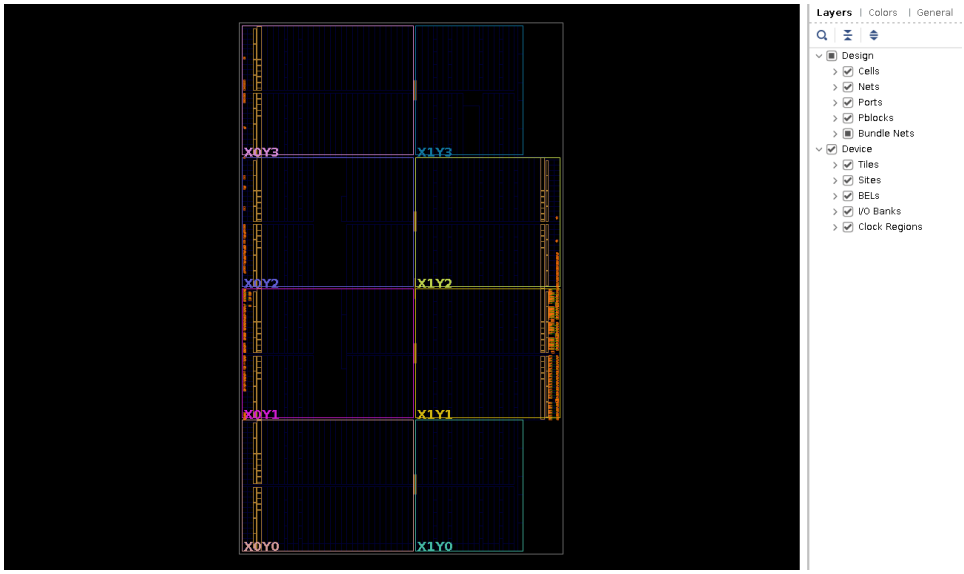
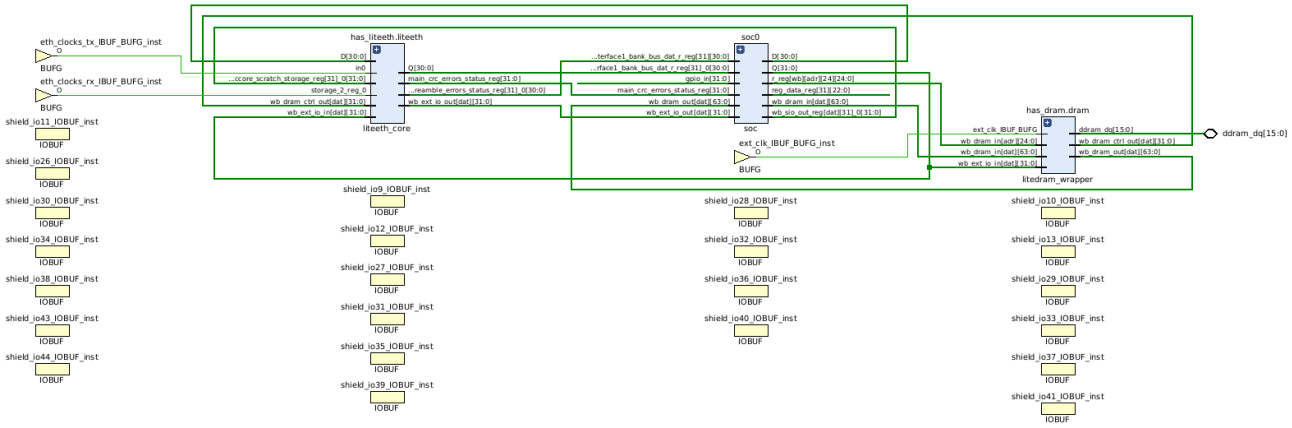


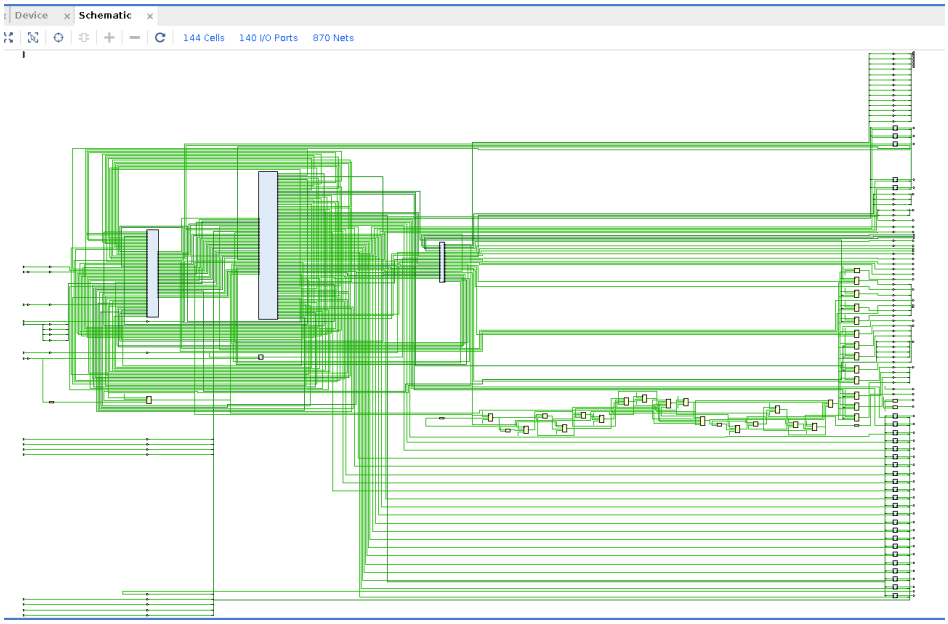
Fig. 10. Synthesized design views for C-based math implementation: (a) Device view showing physical floorplanning

Fig. 10(a) illustrates the synthesized device view for the Artix-7 FPGA. The logic placement is distributed across clock regions X0Y0 to X1Y3, with a noticeable concentration of resources in the central slices. The placement reflects the integration of the Microwatt core with high-level SoC peripherals. The distributed nature of the logic indicates that the standard library implementation consumes a significant portion of the general-purpose fabric to support the complex branching and floating-point emulation required by standard C math routines.



(b) Dataflow design view highlighting SoC-level integration with Ethernet and DRAM controllers

Fig. 10(b) presents the dataflow design view, which reveals a more extensive SoC architecture than the iterative hardware-only versions. This view displays the primary soc0 core interfacing with dedicated infrastructure modules, Ethernet connectivity and for memory management. The hierarchy browser records 29 cells, 16 I/O ports, and 483 nets. The presence of these high-level controllers explains the increased resource utilization observed in the reports, as the system must manage external DRAM access to handle the larger code segments required by the standard C math library. A dense array of shield_io_IOBUF_inst modules ensures robust external signal interfacing.



(c) Synthesis schematic representing the gate-level complexity of the integrated system

Fig. 10(c) provides the synthesis schematic, offering a granular view of the design's logical connectivity. With 144 cells and 870 nets, the schematic is significantly more complex than those of the Taylor series or CORDIC implementations. The interconnected hierarchical blocks represent the deep logic paths needed to execute software-defined transcendental functions. The extensive netlist highlights the overhead of using standard C libraries, where the processor must handle multi-cycle floating-point operations and memory-mapped I/O, resulting in a deeply integrated and logic-heavy design.

xi. Implemented Design Views and Structural Analysis (C-based Implementation)

The physical implementation of the C-based e^x computation system was evaluated post-routing to analyze the structural complexity and spatial distribution of the software-defined arithmetic routines. This implementation represents the final hardware mapping of the Microwatt SoC, including its supporting memory and communication controllers.

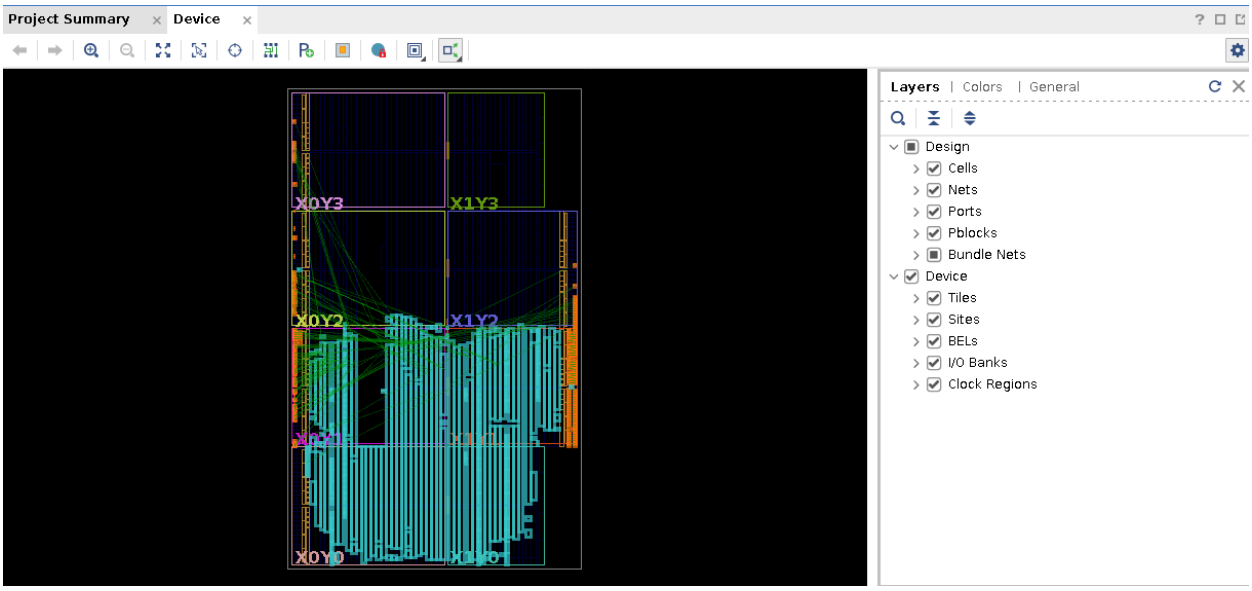
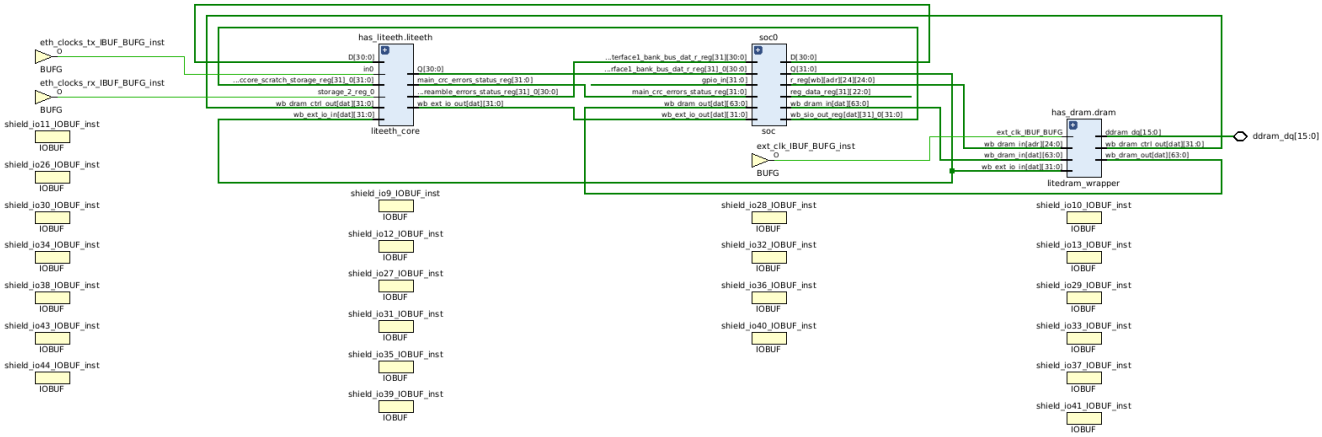


Fig. 11. Post-implementation design views for C-based math implementation: (a) Device layout view showing logic placement and routing density

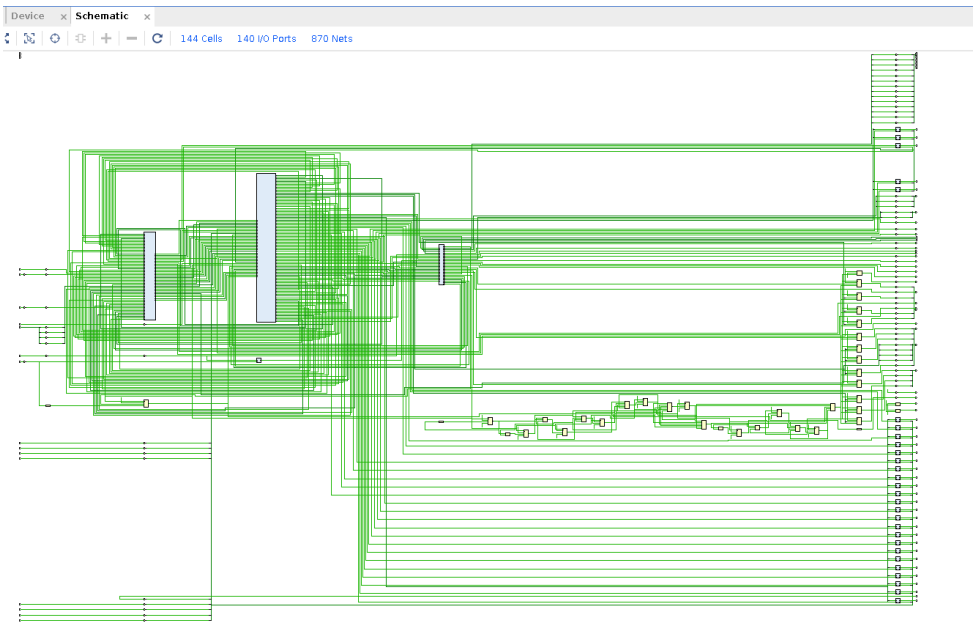
Fig. 11(a) displays the implemented device view, showing the finalized placement of logic elements across the Artix-7 FPGA fabric. The teal-colored regions indicate a dense logic distribution across clock regions X0Y0, X1Y0,

X0Y1, and X1Y1. This clustering suggests that the implementation tools prioritized placing the processor, DRAM controller, and exponential computation paths in close physical proximity to minimize latency. The routing paths demonstrate a well-balanced interconnect network, effectively managing the high net count without creating significant localized congestion, thereby supporting stable execution of the memory-intensive math functions.



(b) Dataflow view illustrating SoC-level connectivity with DRAM and Ethernet modules

Fig. 11(b) presents the implementation dataflow design, highlighting the high-level modularity of the system. The central soc0 core is tightly integrated modules. This architectural configuration is essential for the C-based implementation, as the standard math library routines require external DRAM access for instruction fetching and data storage. The dataflow hierarchy browser reports 29 cells and 483 nets, with a robust I/O interface comprised of 16 ports and multiple shield_io_IOBUF instances, ensuring stable communication between the processor and the physical pins.



(c) Final physical schematic with detailed cell and net statistics

Fig. 11(c) illustrates the implementation schematic, which provides a detailed view of the mapped physical primitives. The schematic features 144 cells and a significantly high net count of 870, reflecting the dense interconnectivity required to support software-driven transcendental operations. This view reveals the complexity of the signal paths that facilitate the interaction between the processor's ALU and the memory-mapped peripherals. The high net density confirms that while the C-based approach simplifies algorithmic development, it results in a more complex physical netlist that requires sophisticated routing to ensure operational integrity.

xii. Timing, Power, and I/O Placement Analysis (C-based Implementation)

To finalize the evaluation of the software-defined e^x computation, a detailed analysis of the physical performance metrics was performed. This assessment focuses on the thermal characteristics, electrical power dissipation, signal integrity via I/O placement, and the temporal reliability of the integrated SoC.

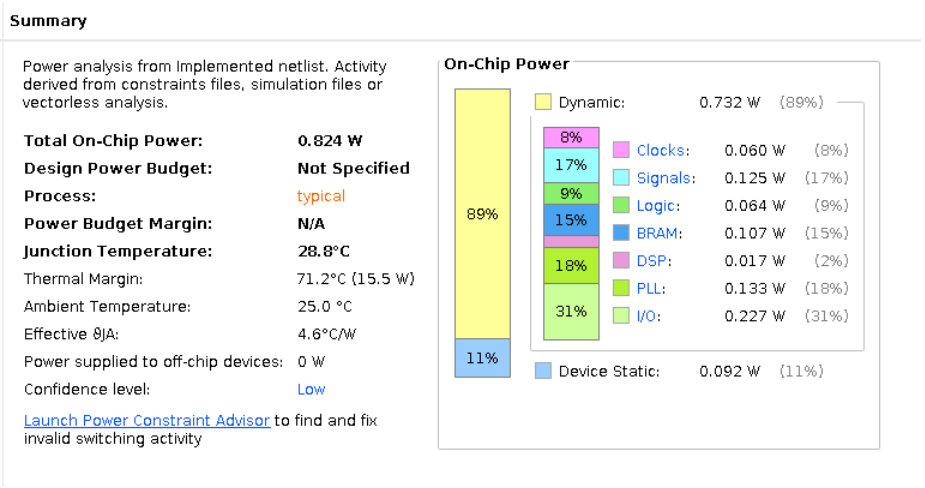
```
------------------------------------------------------------------------------------------------
| Design Timing Summary
| ---------------------
------------------------------------------------------------------------------------------------

    WNS(ns)      TNS(ns)  TNS Failing Endpoints  TNS Total Endpoints      WHS(ns)      THS(ns)  THS Failing Endpoints  THS Total Endpoints     WPWS(ns)     TPWS(ns)  TPWS Failing Endpoints  TPWS Total Endpoints
    -------      -------  ---------------------  -------------------      -------      -------  ---------------------  -------------------     --------     --------  ---------------------  -------------------
      0.358        0.000                      0                43020        0.022        0.000                      0                43020        0.264        0.000                      0                14856


All user specified timing constraints are met.
```

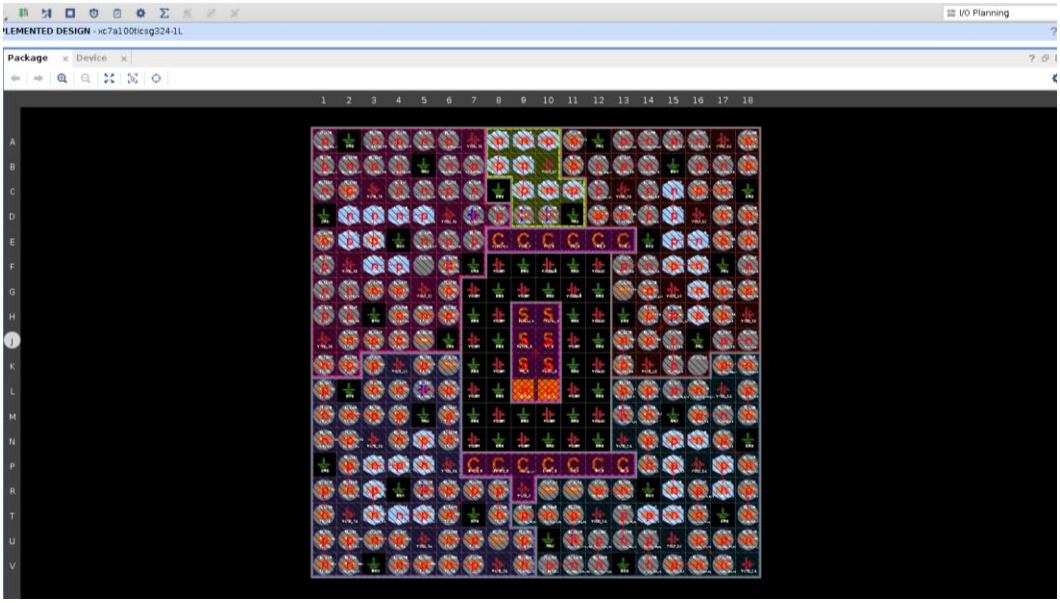
Fig. 12. Performance and Physical Metrics for C-based Implementation: (a) Post-implementation timing summary

Fig. 12(a) provides the design timing summary, confirming that the implementation successfully achieved timing closure. The design maintains a Worst Negative Slack (WNS) of 0.358 ns and a Worst Hold Slack (WHS) of 0.022 ns. The presence of 43,020 total endpoints signifies a much larger and more complex netlist compared to dedicated hardware engines; however, the absence of any failing endpoints (TNS and THS at 0.000 ns) proves that the physical routing can support the required clock frequency. This successful timing verification ensures that the software-based transcendental calculations are executed without data corruption or synchronization errors.



(b) Power summary highlighting the dynamic consumption of SoC peripherals

Fig. 12(b) illustrates the power analysis report, indicating a total on-chip power consumption of 0.824W. The power profile is heavily weighted toward dynamic power (0.732 W, or 89%), which is expected for a processor-driven execution model. Notably, I/O operations and the Clock Manager/PLL account for a combined 49% of dynamic power, reflecting the overhead of the external memory controller and system clocking. BRAM power consumption (15%) is significantly higher than in hardware-optimized implementations, correlating with the larger instruction and data cache requirements of the standard C library. Despite the increased complexity, the junction temperature remains low at 28.8°C, ensuring high thermal reliability.



(c) FPGA package view showing complex I/O pin mapping

Fig. 12(c) presents the package view for I/O placement, detailing the finalized pin assignments on the Artix-7 package. The dense distribution of pins across various banks reflects the complex SoC infrastructure required to support the C math libraries. The systematically organized pin mapping ensures that high-speed data transfers between the Microwatt core and external memory do not suffer from excessive crosstalk, thereby maintaining stable operation during the execution of memory-intensive exponential calculations.

5. RESULTS AND PERFORMANCE ANALYSIS

The performance of the three proposed exponential function implementations—CORDIC-based, C Math-based, and Taylor Series-based was rigorously evaluated on the Microwatt soft-core processor. All designs were implemented on the Xilinx Artix-7 FPGA,The comparative metrics focusing on power consumption, timing performance, and resource utilization are summarized in Table 4.

A. Power Consumption Analysis

The power analysis reveals a significant disparity between hardware-optimized algorithms and software-defined routines. The CORDIC-based implementation emerged as the most power-efficient, consuming a total of only 0.30 W. This efficiency is primarily due to its iterative shift-and-add nature, which minimizes high-frequency switching in complex multipliers. In contrast, the C Math-based implementation recorded the highest power dissipation at 0.824 W, an increase of approximately 174% over the CORDIC approach. This is attributed to the substantial dynamic power (0.732 W) required for software-style library execution, which heavily taxes the I/O and BRAM resources. The Taylor Series-based implementation presented a balanced profile at 0.685 W, with logic-heavy power (0.112 W) reflecting the overhead of multi-term polynomial expansions.

B. Timing and Frequency Performance

Timing closure is critical for ensuring reliable processor operation. All three designs successfully met the timing constraints with zero Total Negative Slack (TNS). However, the CORDIC implementation demonstrated superior timing robustness with a Worst Negative Slack (WNS) of 2.804 ns, providing a
significant safety margin for higher clock frequencies. The Taylor Series and C Math implementations exhibited much tighter timing margins, with WNS values of 0.323 ns and 0.358 ns, respectively. The increased complexity of the Taylor Series netlist (33,010 endpoints) and the C Math netlist (43,020 endpoints) indicates a deeper logic depth, which inherently limits the maximum achievable frequency compared to the hardware-efficient CORDIC engine.

C. Resource and Structural Efficiency

The resource utilization details in Table 4 highlight the architectural trade-offs of each method. The CORDIC implementation required negligible DSP power (~0 W), confirming its suitability for area-constrained FPGAs. Conversely, the Taylor Series method utilized the most DSP resources (recorded at 0.030 W), necessitated by the simultaneous multiplication of high-order polynomial terms. The C Math implementation showed an unusually high I/O power (0.227 W) and BRAM power (0.107 W), which suggests that the standard library calls involve extensive memory-mapped interactions and instruction fetching that are not optimized for the FPGA's internal structural primitives.

TABLE 4 Comparative Performance Summary of e^x Implementations

Parameter	CORDIC-Based	Taylor Series-Based	C Math-Based
Total Power (W)	0.300	0.685	0.824
Dynamic Power (W)	0.195	0.580	0.732
Static Power (W)	0.104	0.106	0.092
WNS (ns)	2.804	0.323	0.358
TNS (ns)	0.000	0.000	0.000
Total Endpoints	37,740	33,010	43,020
Clock Power (W)	0.029	0.047	0.060
Logic Power (W)	0.009	0.112	0.064
BRAM Power (W)	0.033	0.061	0.107
DSP Power (W)	0.000	0.030	0.017
I/O Power (W)	0.005	0.007	0.227

6. DISCUSSION AND CONCLUSION

A. Discussion

The experimental results obtained from the post-implementation analysis in Vivado provide a clear hierarchy of suitability for implementing the exponential function (e^x) on an FPGA-based soft-core processor. The evaluation highlights the inherent trade-offs between computational complexity, resource occupancy, and physical performance.

The CORDIC algorithm is the definitive winner for hardware-constrained and high-performance embedded systems. Its reliance on an iterative shift-and-add architecture rather than dedicated multipliers yields the highest timing headroom (WNS of 2.804 ns) and the lowest power footprint (0.30 W). The absence of DSP slice utilization ensures that this implementation is highly portable across various FPGA families, including low-cost devices where arithmetic primitives are limited. The wide timing margin further suggests that the CORDIC engine could be clocked significantly higher than the base Microwatt frequency, potentially supporting multi-core or high-throughput DSP pipelines.

The Taylor Series method serves as a pragmatic architectural middle ground. By utilizing dedicated DSP slices for polynomial expansion, it provides a more straightforward path to increasing mathematical precision compared to CORDIC; a designer can simply increase the order of the series to reduce approximation error. However, this flexibility comes at the cost of higher logic density and a power consumption of 0.685 W. The relatively tight timing slack (0.323 ns) indicates that the deeply nested multiplication and addition paths required for the higher-order terms represent the critical path of the custom instruction.

Finally, the C Math-based implementation, while offering the greatest ease from a software development perspective, is the least efficient for hardware deployment. The high total power (0.824 W) and the significant overhead in BRAM and I/O power usage indicate that mapping standard software functions directly to hardware via the processor is sub-optimal. This approach essentially forces the Microwatt core to manage a large

instruction footprint and frequent memory-mapped interactions, leading to inefficiencies that are not present in the dedicated hardware-centric pipelines.

B. Conclusion

In this research, three distinct architectural approaches for implementing the exponential function (e^x) on a Microwatt soft-core processor were designed, synthesized, and implemented. By integrating these functions as custom instructions within the OpenPOWER-compliant ISA, the computational burden was shifted from general purpose software execution to optimized hardware-accelerated stages.

Based on the rigorous post-implementation analysis, the CORDIC-based approach is recommended as the primary solution for FPGA-based acceleration. It provides a superior power-to-performance ratio, consuming 63% less power than the software-style C implementation while maintaining the highest timing reliability. For applications where mathematical precision must be tuned dynamically or where DSP resources are abundant, the Taylor Series expansion remains a viable and flexible alternative.

This study concludes that for transcendental function acceleration on FPGAs, dedicated hardware-centric algorithms like CORDIC significantly outperform software-to-hardware mapped library functions across all critical physical metrics. The successful integration of these units as ISA extensions demonstrates a robust methodology for enhancing the performance of open-source RISC architectures in scientific and real-time signal processing domains. Future work will involve extending this comparative framework to other transcendental operations, such as logarithmic and trigonometric functions, to create a comprehensive hardware-accelerated mathematical library for the Microwatt platform.

7. REFERENCES

[1] J. Sudha, M. C. Hanumantharaju, V. Venkateswarulu, and H. Jayalaxmi, "A Novel Method for Computing Exponential Function Using CORDIC Algorithm," Procedia Engineering, vol. 30, pp. 519–528, 2012.

[2] S. Vaidya, Implementation of Hyperbolic CORDIC with AXI Full Interface, International Journal of Engineering Research & Technology (IJERT), vol. 9, no. 6, pp. 231–235, Jun. 2020. DOI: 10.17577/IJERTV9IS060190

[3] J. S. Walther, "A Unified Algorithm for Elementary Functions," in Proc. Spring Joint Computer Conference, Atlantic City, NJ, USA, 1971, pp. 379–385.

[4] D. R. Llamocca-Obregón and C. P. Agurto-Ríos, "A Fixed-Point Implementation of the Expanded Hyperbolic CORDIC Algorithm," Latin American Applied Research, vol. 37, no. 1, pp. 83–91, 2007.

[5] N. Simmonds, J. Mack, S. Bellestri, and D. Llamocca, "CORDIC-Based Architecture for Powering Computation in Fixed-Point Arithmetic," arXiv preprint arXiv:1605.03229, 2016.

[6] A. Vaishnav, K. D. Pham, and D. Koch, "A Survey on FPGA Virtualization," in Proc. 28th International Conference on Field-Programmable Logic and Applications (FPL), Dublin, Ireland, 2018, doi: 10.1109/FPL.2018.00031.

[7] F. Restuccia, A. Biondi, M. Marinoni, G. Cicero, and G. Buttazzo, "AXI HyperConnect: A Predictable, Hypervisor-Level Interconnect for Hardware Accelerators in FPGA SoC," in Proc. 57th ACM/IEEE Design Automation Conference (DAC), 2020, doi: 10.1109/DAC18072.2020.9218652.

[8] P. Nilsson, A. U. R. Shaik, R. Gangarajaiah, and E. Hertz, Hardware Implementation of the Exponential Function Using Taylor Series, 2014 NORCHIP Conference, Tampere, Finland, pp. 1–4, Oct. 2014. DOI: 10.1109/NORCHIP.2014.7004740

[9] M. H. Quraishi, E. B. Tavakoli, and F. Ren, "A Survey of System Architectures and Techniques for FPGA Virtualization," arXiv preprint arXiv:2011.09073, 2020.

[10] D.-C. Le and C.-H. Youn, "Criteria and Approaches for Virtualization on Modern FPGAs," arXiv preprint arXiv:1904.03838, 2019.

[11] R. Rekha and K. P. Menon, FPGA Implementation of Exponential Function Using CORDIC IP Core for Extended Input Range, IEEE

[12] R. Andraka, "A Survey of CORDIC Algorithms for FPGA Based Computers," Proceedings of FPGA '98, pp. 191–200, 1998.

[13] P. K. Meher et al., 50 Years of CORDIC: Algorithms, Architectures, and Applications, IEEE.

[14] Y. H. Hu, CORDIC-Based VLSI Architecture for Digital Signal Processing.

[15] STMicroelectronics, Coordinate Rotation Digital Computer (CORDIC) Algorithm to Compute Trigonometric and Hyperbolic Functions, Design Tip DT0085, STMicroelectronics, Geneva, Switzerland, 2020

[16] OpenPOWER Foundation, Power ISA Version 3.1 Specification. OpenPOWER Foundation, 2020.

[17] Hofstee-HP, MicroWatt on Arty A7-100T: Custom Ops Example, GitHub Repository. Available: https://github.com/hofstee-hp/MicroWatt_on_Arty_A7-100T/tree/main/Custom_Ops_Example

[18] A. Blanchard, Microwatt, GitHub Repository. Available: https://github.com/antonblanchard/microwatt

[19] Xilinx Inc., Vivado Design Suite User Guide: Power Analysis and Optimization (UG907), Xilinx, San Jose, CA, USA, version 2019.2, 2019. Available: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2019_2/ug907-vivado-power-analysis-optimization.pdf

[20] Xilinx Inc., 7 Series FPGAs Data Sheet: Overview (DS180), Xilinx, San Jose, CA, USA, v2.6, 2018. Available: https://www.xilinx.com/support/documentation/data_sheets/ds180_7Series_Overview.pdf